

Pathfinding Algorithm for Drone Delivery Using Branch and Bound Method

cs-24-sw-1-p1-05

Anders Mathias Larsen, Daniel Sloth, Jacob Christian Larsen, Lena Hayes,
Philip Vestergaard Hellerup Jørgensen, Philippe Christian Frøkjær Bourrachot
and Viktor Alexander Parkhøi





AALBORG UNIVERSITY
STUDENT REPORT

Computer Science

Aalborg University

<http://cs.aau.dk>

Title:

Pathfinding Algorithm for Drone Delivery
Using Branch and Bound Method

Project Period:

Fall Semester 2024

Project Group:

cs-24-sw-1-p1-05

Participants:

Anders Mathias Larsen

Daniel Sloth

Jacob Christian Larsen

Lena Hayes

Philip Vestergaard Hellerup Jørgensen

Philippe Christian Frøkjær Bourrachot

Viktor Alexander Parkhøi

Supervisor:

Daniele Toller

Date:

2026-08-19

Copies: 1

Page Numbers: 50

Abstract:

This report explores the use of drones for optimizing last-mile medical supply deliveries to rural areas, addressing healthcare accessibility challenges. It focuses on implementing the Branch and Bound algorithm to solve the Traveling Salesman Problem (TSP) for drone routing. The proposed solution reads delivery point coordinates from a file, calculates Euclidian distance between nodes, and applies the Branch and Bound method to determine the shortest delivery path.

The report covers problem context, methodology, program design, implementation, and testing results. Additionally, it identifies limitations and suggests future improvements, including alternative algorithms, integration of charging stations, and enhanced user experience features. This work demonstrates the potential of drone delivery to improve healthcare access in underserved rural communities, particularly during lockdowns.

Contents

1	Introduction	1
2	Problem Analysis	2
2.1	Healthcare Challenges in Rural America	2
2.1.1	Distance to Healthcare Providers	2
2.1.2	Lack of Healthcare Practitioners	2
2.2	How can Drone Delivery be used in the Healthcare System for Specifically Rural and Suburban Communities?	3
2.2.1	Current Applications for Drones in Medical Supply Deliveries	3
2.3	COVID-19 in Rural America	4
2.3.1	Impact of the COVID-19 Pandemic in Rural Areas	4
2.3.2	Testing Infrastructure in America During the COVID-19 Pandemic	4
2.3.3	Testing Methods	4
2.4	How UAV Technology was used during the COVID-19 Pandemic?	5
2.4.1	Deployment of Drones for COVID-19 Medical Deliveries	5
2.4.2	Leveraging Drones for Improved Testing Access in Rural Areas	5
2.5	UAVs in Relation to Other Last-Mile Delivery Modalities	6
2.5.1	Energy Efficiency	7
2.5.2	Cost Efficiency	8
2.6	Challenges with Drone Delivery	9
2.6.1	Battery & Charging	9
2.6.2	Regulations	9
2.6.3	Social Perception	10
3	Problem formulation	12
3.1	Problem Delimitation	12
3.2	Problem Statement	13
4	Methodology & Theory	14
4.1	Tools & Technologies	14
4.1.1	Programming Language	14
4.1.2	IDE	14
4.1.3	Version Control	14
4.2	Graph Data Structure & Algorithms	14
4.2.1	Graph Theory	14
4.2.2	Graph Representation Using Adjacency Matrices	15
4.2.3	The Traveling Salesman Problem & The Vehicle Routing Problem	17
4.2.4	The difference between TSP & VRP	18
4.2.5	Euclidean Distance & Graph Creation	18
4.2.6	Branch & Bound Method	20
5	Program design	21
5.1	Solution Constraints & Limitations	21
5.1.1	TSP or VRP	21
5.1.2	Input Size & Complexity	21
5.1.3	Effect from Wind & Weather	21
5.1.4	Battery, Charging & Weight Limitations	21
5.2	Product Requirements	22
5.2.1	Functional & Non-functional requirements	22
5.2.2	MoSCoW	23

5.3	Program Structure	24
5.3.1	High-level Flowchart	24
5.4	Algorithm design	25
5.4.1	Importing Graph From a File	25
5.4.2	Edge Weight Algorithm	26
5.4.3	Branch & Bound Algorithm	27
5.5	Implementation	37
5.5.1	Node struct and Node array	37
5.5.2	Reading nodes from file	38
5.5.3	Calculating edge weights	39
5.5.4	Pathfinding using branch and bound method	40
5.5.5	Output	45
6	Discussion & Future Developments	46
6.1	Discussion	46
6.2	Scalability	46
6.3	Other Applications	47
6.4	Further Developments	47
6.4.1	Incomplete graphs	47
6.4.2	Loading bar	47
6.4.3	Graphical output	47
6.4.4	Charging station and truck-supported delivery	47
7	Conclusion	49
	Bibliography	50
9	Attachments	53
9.1	Example Breakdown of the Branch & Bound algorithm on TSP on a Graph	53
9.2	Part 135 certificate tiers	57
9.3	General rules for operating a UAS for both commercial recreational use	57
9.4	Graph Representation Using Adjacency Lists	58
9.4.1	What is an Adjacency List?	58
9.4.2	Memory Usage	58
9.4.3	Time Complexity for Edge Lookups	58
9.5	Graph Representation Using Edge Lists	59
9.5.1	What is an Edge List?	59
9.5.2	Memory Usage	59
9.5.3	Time Complexity for Edge Lookups	59
9.6	Random node generation	60
9.7	Brute Force Time Comparison	61

Preface

The project is made by group cs-24-sw-1-p1-05, a 1. semester software group from Aalborg University. The project's main theme is "Package Delivery" and the goal is to make a program in C that solves a problem. This project was produced with guidance from, Daniele Toller, a supervisor at Aalborg University. The finished program can be found at <https://github.com/PhilipHellerup/P1-Project-Program-In-C.git>

Aalborg University, December 20, 2024

Anders Mathias Larsen
amla24@student.aau.dk

Daniel Sloth
dsloth23@student.aau.dk

Jacob Christian Larsen
jcla24@student.aau.dk

Lena Hayes
lhayes24@student.aau.dk

Philip Vestergaard Hellerup Jørgensen
pjorge24@student.aau.dk

Philippe Christian Frøkjær Bourrachot
pbourr24@student.aau.dk

Viktor Alexander Parkhøi
vparkh24@student.aau.dk

Access to healthcare services remains a significant challenge in rural areas, particularly in the United States. Geographic isolation, limited infrastructure, and workforce shortages limit timely access to medical care. The COVID-19 pandemic has further emphasized the critical need for innovative solutions to address these inequalities.

Unmanned aerial vehicles (UAVs) are emerging as a solution to overcome logistical barriers to healthcare delivery. Drones can overlook traditional transportation obstacles such as traffic congestion, and poor road networks, and are more effective in transporting fewer and smaller packages than conventional delivery trucks [1]. This unique capability makes drones ideal for last-mile delivery of essential medical supplies, especially in underserved and isolated rural areas.

Efficient routing is a key factor, that ensures the success of drone delivery operations, and optimization algorithms can play a vital role in reducing travel times and operational costs. This report explores the use of advanced routing algorithms, focusing on the Branch and Bound method, to solve the Traveling Salesman Problem (TSP) for drone delivery. The goal is therefore, to optimize delivery routes by minimizing the total distance, while ensuring that drones can visit all delivery points exactly once and return to the starting location as quickly and efficiently as possible.

The project focuses on implementing a software solution that minimizes delivery routes based on geographical coordinates. The following sections will provide insight into the problem context, methodology and theory, program design, and implementation. Through this, the report aims to demonstrate how algorithm-driven drone routing can improve healthcare options in rural areas, particularly during health crises or pandemics.

2.1 Healthcare Challenges in Rural America

Over 51 million Americans live in rural areas. The United States Census Bureau defines rural areas as areas holding less than 2,000 housing units or fewer than 5,000 people [2]. There is clear evidence for discrepancies in access to healthcare in rural areas, compared to urban areas. Rural areas have higher levels of chronic disease, poor mental and physical health, as well as limited access to digital healthcare solutions [3]. This section will highlight some of the most important barriers to healthcare in rural areas and how rural areas were impacted during the COVID-19 pandemic.

2.1.1 Distance to Healthcare Providers

Simply getting to a doctor can be a problem in rural areas. Patients are less likely to travel to see a doctor the further away they live and in some areas, accessing healthcare is almost impossible due to the extent of travel [3], [4]. Owning a car does not significantly affect attendance, but patients with a driver's license are at least twice as likely to attend appointments compared to those who do not possess a driver's license [3].

One solution to this may be to bring healthcare to the patient directly. Patients having difficulties accessing prescription medication are trying to overcome this by ordering pharmaceuticals via mail and saving a trip to the pharmacy. Sometimes, healthcare providers offer travel services to patients in need who cannot travel to see a doctor themselves [3].

2.1.2 Lack of Healthcare Practitioners

Adding to the problem, a shortage of practitioners in rural areas makes it even more challenging to get medical help. One study found that for patients with similar socioeconomic standing, insurance status, and demographic characteristics, rural patients had less access to mental health services compared to urban populations [5]. Additionally increasing urbanization and the shortage of healthcare workforce will further extend the severity of challenges in rural areas. It has become evident that new innovative solutions are needed [6].

2.2 How can Drone Delivery be used in the Healthcare System for Specifically Rural and Suburban Communities?

Rural communities are often challenged, when it comes to keeping a medical staff with a proper amount of healthcare professionals, like doctors and nurses. This results in older generations and citizens living in suburban- and rural areas, getting provided with health services worse than what is available in urban areas [7]. In cases like this, it is important to try and improve the quality of healthcare, but also the access to high-quality medical supplies, like vaccines, blood samples, pills, bandages, etc. Drones can, in this scenario, play an important role, when it comes to fast and efficient delivery of medical supplies. Delivering medications via drones, could increase the accessibility of medications and therefore improve healthcare in remote areas [7].

In [7], the authors studied how drone-assisted delivery of medical supplies could improve healthcare systems in remote communities. In [7] it is indicated that drone-assisted systems can overcome geographic and other delivery problems in remote areas by allowing quicker delivery methods like drones, which don't face delays, because of traffic, environment, and limited infrastructure. The authors argue that drones paired with recharging stations could extend the range, enabling them to cover longer distances. The article also highlights how trucks often act as mobile depots, carrying drones closer to the target delivery zones, after which the drones can fly to remote locations while accessing recharging stations strategically placed along the drone's paths/routes. The combination of trucks and drones effectively minimizes delays, especially in medical emergencies, and also reduces transportation costs by decreasing the dependency on truck-only deliveries [7].

In summary, integrating drones with recharging stations into healthcare logistics can expand and enhance access to crucial healthcare services, like medical supplies, in remote areas, while also having fast and efficient deliveries.

2.2.1 Current Applications for Drones in Medical Supply Deliveries

Some of the current applications for drones in healthcare have comprised deliveries containing blood samples, medicine, defibrillators, and vaccines. A startup in the United States, named *Zipline*, has begun using drones to deliver medical supplies, containing blood samples, to remote areas of Rwanda [8]. In 2019 *Zipline* used UAVs (Unmanned Aerial Vehicles) to hand out life-saving medical supplies, like blood and vaccines to around 2000 healthcare centers/hospitals in the south of Ghana [7]. There are also examples of other companies using drone/UAV technology to deliver medical supplies, such as the aerial drone company, *Matternet*. *Matternet* has flown 1800 flights over Swiss cities, where they delivered 850 blood- and body tissues in Lugano and Bern. Later they also expanded their operations/deliveries to Zurich and Haiti, where they deliver medications [7]. This shows that drones are and can be used in scenarios, where people require medical supplies.

2.3 COVID-19 in Rural America

The COVID-19 pandemic posed significant challenges across society, disrupting routines, isolating communities, and generating widespread loss and uncertainty. While urban centers generally had more resources to mitigate the pandemic's impact, rural areas were more affected, often struggling with limited healthcare access, fewer support services, and greater social isolation [9].

2.3.1 Impact of the COVID-19 Pandemic in Rural Areas

A study [10] shed some light on the pandemic's impact on rural compared to urban areas. They profiled the pandemic's impact on those living in the rural American West and found that rural areas were more negatively impacted in almost every way. This included employment, overall life satisfaction, mental and physical health, and economic outlook. These findings were consistent across age, sex, ethnicity, and education level. The researchers showed that people living in rural parts of America feel the dramatic impacts of the pandemic just as much and in some cases even more than those in urban areas. As mentioned in Section 2.1, rural regions have limited access to healthcare facilities as well as more vulnerable labor markets, which are highly susceptible to pandemic-related closures, making the road to recovery even more difficult for these areas. Additionally, the authors found that rural areas had far more limited access to COVID-19 testing, which is very important to mitigate virus spreading.

2.3.2 Testing Infrastructure in America During the COVID-19 Pandemic

The COVID-19 pandemic highlighted major areas for improvement in the United States' testing infrastructure. One significant issue was that the system struggled to meet demand, particularly in remote and underserved areas, due to limited access to traditional testing sites and delays in processing test results. The rapid establishment of Community-Based Testing Sites helped address access gaps and although mobile testing units were deployed in some areas, they were not sufficient to cover the vast geographic and demographic disparities across the country [11]. This highlighted the need for innovative solutions to speed up testing and delivery of supplies to high-demand areas [12].

2.3.3 Testing Methods

Population-wide testing was used as a method to manage the spread of the virus during the COVID-19 pandemic. With sufficient broad testing of an entire population, it was considered possible to identify and isolate most of the infected individuals. However extensive testing is very costly and multiple technical and practical limitations made it ineffective to implement nationwide [13].

One article [13] analyzed the effectiveness of large-scale testing. They argued that testing an entire population was too expensive and ineffective as most positive results are false positives. Instead, testing should be reserved for individuals having symptoms, those closely related to infected individuals, and populations in high-risk groups with an increased probability of infection. This approach has proven effective in coordination with other interventions, such as targeted regional lockdowns and travel restrictions.

2.4 How UAV Technology was used during the COVID-19 Pandemic?

During the COVID-19 pandemic, various organizations, including governments and private companies, turned to drone and unmanned aerial vehicle (UAV) technology to help tackle the challenges of the crisis. In [14], they argued that drones could be used to pick up and deliver lab samples and medical supplies. By using drones for this task, healthcare providers could speed up testing and reduce the risk of exposure to the virus.

2.4.1 Deployment of Drones for COVID-19 Medical Deliveries

During the COVID-19 pandemic, eighteen countries deployed drones to assist deliveries of supplies. Some countries did it as a test, while others chose to uphold their drone delivery operations. Rwanda, Ghana, and Malawi chose from the beginning of the pandemic, to use drones to deliver COVID-19 supplies and COVID-19 samples, given that the three countries already had drone operations before the COVID-19 pandemic, as mentioned in Section 2.2.1. These three countries reported that the benefits of using drones in the context of COVID-19 are, the speed of the delivery, the extended transportation network reach for the last mile deliveries, and the limited physical contact and reduced risk of infection during delivery [14].

2.4.2 Leveraging Drones for Improved Testing Access in Rural Areas

Drone delivery minimizes human involvement, allowing packages to be transported without direct human-to-human contact. This approach reduces the risk of virus transmission and contamination, especially in healthcare settings where limiting physical interaction is crucial. By delivering items directly to secure drop zones, drones provide a safer and more efficient alternative to traditional delivery methods.

This implies that drones could be crucial in overcoming the logistical challenges of reaching rural areas and ensuring equal access to healthcare services, including testing during pandemics. Rural areas in the U.S. often face significant barriers to accessing healthcare, from fewer local testing facilities to long travel times for essential health services. This was especially problematic during the COVID-19 pandemic, where remote populations faced delays in testing and result processing. Drones offer a way to bridge these gaps by delivering testing kits and transporting collected samples, thereby reducing wait times and travel burdens for residents in isolated areas [12].

With rapid delivery of test kits and medical supplies, bypassing the limitations of road transport in remote or geographically challenging areas. For instance, drones could deliver self-administered testing kits directly to residents, allowing them to test at home. This approach would reduce the need for travel, minimizing both inconvenience for patients and potential virus exposure. Furthermore, drones could collect completed test samples from designated collection points, returning them to labs faster than traditional methods and decreasing turnaround times [12].

The adoption of drones in rural healthcare could significantly minimize accessibility issues, making it possible to deliver faster, more efficient testing solutions to underserved communities. As technology and regulatory frameworks evolve, the potential for drones to enhance public health response in rural areas grows, offering a promising path to strengthen the resilience of America's healthcare infrastructure for future emergencies [11].

2.5 UAVs in Relation to Other Last-Mile Delivery Modalities

The final stage of package delivery, commonly referred to as “last-mile delivery,” plays a critical role in the logistics process. This phase is particularly significant as it directly influences customer satisfaction and timely delivery. Last-mile delivery is recognized as one of the most cost-intensive parts of the overall supply chain, due to the complex logistics involved in reaching individual consumers efficiently.

A global study [15] revealed a substantial rise in customer demand for high-quality service. The findings also indicated a growing preference for personalized services, with consumers expecting flexible delivery options, including choices for delivery time and location. Additionally, there has been an increase in demand for more versatile solutions in the collection and return of parcels, highlighting the need for more adaptable logistics practices to meet customer expectations.

Integrating drones into delivery logistics, particularly in last-mile delivery, has the potential to streamline the delivery process. However, to effectively incorporate drones in last-mile delivery, it is essential to examine both the potential challenges and advantages of this technology.

According to [16], it is estimated that more than 55% of the world’s population lived in urban residences in 2018 and by 2030 urban areas are expected to contain up to 60% of people globally. With the increase in the density of people living in urban areas, society faces new challenges. One of the challenges is related to mobility, which is becoming an even greater problem, with the rise of e-commerce businesses like *Amazon* and *Alibaba*. According to [1], a global shipping and mailing company, there were 87 billion packages delivered globally in 2018 and it is expected that by 2025, the volume of packages will increase to 200 billion. It is also estimated that around 86% of *Amazon’s* packages weigh less than 2.27 kg, which makes it possible for drones to deliver these sorts of packages. Therefore it seems natural to explore how drone delivery can be more environment-friendly and cost-efficient compared to other traditional delivery methods.

2.5.1 Energy Efficiency

Drones are being explored more and more, regarding their potential to improve and enhance the last-mile delivery efficiency compared to other traditional delivery methods, such as trucks and vans. A study [17] has shown that drones, mainly quad-copters, provide significant energy savings and less greenhouse emissions. The study revealed that drones can use up to 94% less energy per package delivery than traditional delivery vehicles, especially when it comes to packages of a smaller size and high-value items/products, such as electronics and medical supplies. The study showed how drones are an appealing option, given their fast and environmentally friendly deliveries.

The study also showed a chart, as shown in Figure 1, which compares different types of delivery methods, including medium-duty trucks, small vans, drones, and e-cargo bicycles, showcasing their corresponding energy consumption per package, on the left side, and carbon emissions per package on the right side. The different colors indicate the sources of emissions, like upstream fuel, battery life cycle, and fuel consumption [17].

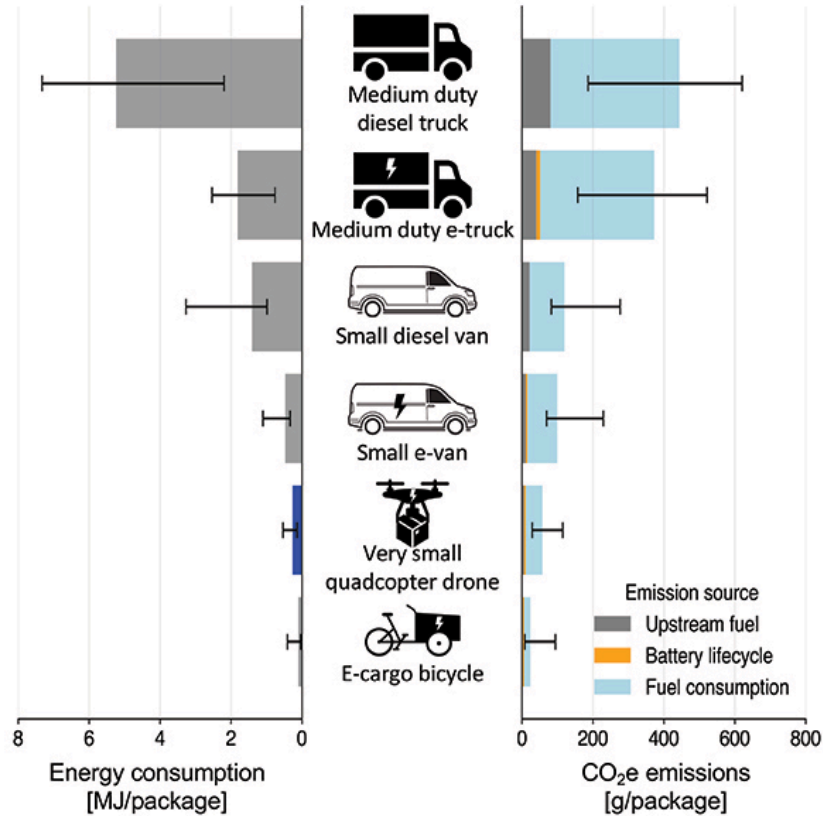


Figure 1: A comparison of last-mile delivery methods, by energy consumption and CO₂ emissions showed that very small quad-copter drones and e-cargo bikes are most efficient [17].

2.5.2 Cost Efficiency

An assessment analysis [1] is shown in Figure 2. The analysis is based on a real scenario, where around 60.000 parcels needs to be delivered to different locations in the capital, Brussels. The analysis shows the cost assessment of parcel delivery using e-vans vs drones as a delivery method. The analysis showed that for same-day deliveries, e-vans are more cost effective, due to the cost per delivery being around 0.14 euros, while drones cost an estimated 0.59 euros. The result of this analysis makes it clear that the higher capacity of the e-vans is crucial to its cost efficiency. However if there is a scenario, where a delivery has to be within 30 minutes from the order, drones become more of a contender. The analysis indicates that in that kind of situation, the cost per delivery per drone is expected to be 1.00 euros, while the cost for the e-van rises to 4.35 euros. The result of this analysis indicate that drones might not replace all of the last-mile parcel deliveries, but it can definitely be used to cover a niche market, where products need to be delivered, within a short amount of time [1].

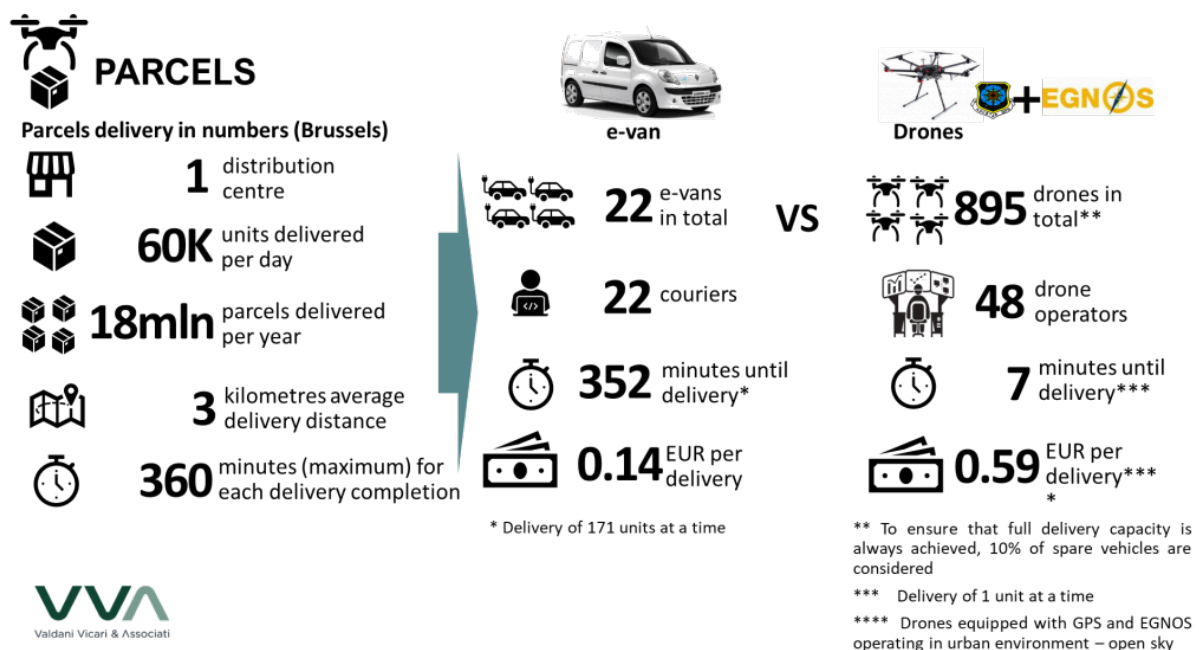


Figure 2: Cost assessment analysis of parcel delivery using e-vans vs drones [1].

2.6 Challenges with Drone Delivery

Drone delivery presents a multitude of challenges, similar to those encountered in other delivery methods. Among the primary issues are battery life and payload capacity. Due to their compact design, drones cannot generally carry large batteries or substantial payloads, which can complicate delivery processes and limit their operational efficiency. Furthermore, regulations and public perception are critical factors in the integration of new technologies in society.

Despite numerous challenges associated with drone technology, there are also considerable advantages. Routing with drones is simplified, as drones can travel in a direct path to their destination, as well as avoid road traffic. This enables them to reach their target location quicker than ground vehicles, enhancing delivery speed and efficiency [18].

Analyzing the challenges will contribute to defining a comprehensive problem framework for drone delivery, as well as identifying potential advantages inherent to this technology.

2.6.1 Battery & Charging

Battery life is one of the major limitations in drone delivery, restricting both range and payload capacity. Additionally, short-term battery life limits the drones to operate within a more narrow radius, which confines deliveries to areas near distribution centers. This range restriction becomes even more critical as the drones will be forced to save energy for the return trip back to the center. Establishing a network of charging points could solve this problem by allowing drones to recharge on the route, extending their operational range and enabling them to carry both more and heavier packages. By overcoming battery limitations, charging points would make drone delivery faster and accessible to a broader range of locations.

Creating a network of charging points could fundamentally improve the operational scope of drones used for delivery, but this approach requires careful planning and infrastructure investment. Strategically placed charging stations would enable drones to recharge mid-route, addressing the current limitations on range and payload capacity. Such a network would not only allow for deliveries over longer distances but also enable drones to carry heavier loads, making them more effective for healthcare-related logistics, such as the transport of testing kits and medical supplies to remote or underserved areas.

The implementation of charging points and enhanced battery technology would significantly impact public health logistics, particularly in times of crisis like pandemics. With a broader operational range, drones could deliver essential supplies to more remote areas that traditional delivery systems cannot reach promptly. This would ensure that rural and underserved communities receive timely access to COVID-19 tests, vaccines, and other critical medical supplies, helping to reduce disparities in healthcare access and response. By solving the limitations of battery life, this infrastructure would not only improve drone delivery capabilities during pandemics but also create a long-term solution for future public health responses. This innovation could play a vital role in establishing a more resilient healthcare supply chain prepared to meet the demands of both routine and emergencies.

2.6.2 Regulations

Within the realm of drones. In both commercial use and recreational use, there are several regulations and rules that one would need to follow to utilize them safely and in accordance with the laws at any given place in the world. This report will take a look at laws around the world, but will mainly focus on the laws and regulations in Europe and the United States of America.

In the United States of America specifically, one is required to have a Part 135 certification for the drone in question to be allowed to carry the property of another beyond visual line of sight, which is not normally allowed, as operating a UAS (Unmanned Aerial System) beyond visual line of sight is illegal without the Part 135 certification [19].

There are four tiers to a Part 135 certificate. The tiers can be found in Section 9.2 under attachments.

In addition to having the part 135 certificate, a UAS pilot must also follow certain general rules and restrictions that hold for all UAS pilots, both recreational and commercial. These rules and regulations can be found in Section 9.3 under attachments.

All of these rigorous regulations, waivers, and certificates are essential for conducting complex drone operations, such as beyond visual line of sight (BVLOS) flights. The process are rigorous, costly, and time-consuming, involving multiple phases to ensure safety and compliance. For instance, a company called Drone Express began its Part 135 application over three years ago as of 2022, highlighting the lengthy nature of the process. [20]

The part 135 certificate was not invented for drone delivery but rather mainly for smaller airlines/air carriers to perform passenger flights or cargo transportation flights. It later got modified granting exceptions on rules that were not well applied for unmanned drone operations, these modifications have happened on a case-by-case basis as the FAA evaluates individual applications [21], this might have been seen as an annoyance, leading to the FAA's new initiative called the FAA Reauthorization Act of 2024, it is a legislation that is been introduced to congress on two occasions 13/06-2023 and 29/02-2024 with its most important points in terms of the project is "requiring the FAA to establish a regulatory pathway for the certification or approval of commercial unmanned aircraft (i.e., drones) to operate beyond the visual line of sight." [22] Nothing yet has become law, yet it highlights their proposal for a new regulatory pathway specifically designed for commercial drones, which could allow for more tailored requirements focused on drones, potentially making BVLOS certification more efficient, achievable and encouraging faster development of drone services across industries like healthcare, logistics, and public safety.

2.6.3 Social Perception

Privacy concerns stand as a big barrier to the public's acceptance of drones as a delivery method, according to survey findings presented in [23]. These concerns largely stem from a lack of comprehensive knowledge about drone technology, which influences how people perceive the risks associated with them. The studies knowledge section reveals that the general public often learns about drones primarily through mainstream media and movies. These sources frequently highlight military or surveillance applications, leading many to associate drones with privacy intrusion and security risks.[23], in the article made in 2020, the perception of drones through their survey conducted in the U.S didn't change people's perception when met with info showing both the benefits and risk associated with drones, but in some cases, it did change a little based on the knowledge giving in specific applications of the drone e.g public support for cargo use increased from 37% to 52% when participants were given details about the benefits. The article also suggests a way for the public to gain a more informed insight into drones and the different distinctions between types of drones and their applications. Where there may be big differences in the surveys, public knowledge, and regulations from the U.S to other countries there might have been an increase in content surrounding drones, with the increase of commercial use of drones with examples in Rwanda and Ghana, where *Zipline* delivers medical supplies to remote

areas [7], and Japan exploring drone use for parcel deliveries to rural and mountainous areas [24]. This may have had an influence and an underlying change in people’s perception from 2020 to 2024 with it being highlighted in the Finnish study that concludes a much more positive and neutral opinion [25].

The Finnish study on public acceptance of automated last-mile delivery systems, as outlined in [25], provides important insight into how drones are perceived. A survey with 500 respondents results showed, that most had either a positive or neutral attitude towards the technology. Key factors contributing to this acceptance, included affordability, efficiency, and environmental factors, all of which were highlighted as benefits by the respondents of the survey. Notably, nearly half of the participants emphasized the importance of environmental friendliness in drone-based deliveries, signaling that the public values the ecological benefits drones may offer.

However, the survey also revealed differences in acceptance based on geographic location. For example, respondents from suburban and moderately populated areas were more inclined to favor drones over delivery robots, compared to those in city centers or inner cities, where the enthusiasm for drones was somewhat lower. This suggests that public trust in drones is influenced by the specific context in which they operate, with urban residents possibly more skeptical due to factors like congestion, living in apartment buildings making delivery way harder, or unfamiliarity with the technology [25].

Overall, while the survey data points to generally positive attitudes, the results highlight the need for further attention to local contexts and individual preferences in getting public acceptance of drone deliveries. Trust remains a crucial factor in ensuring the widespread integration of drones in everyday logistics [25].

3

Problem formulation

3.1 Problem Delimitation

This section will define the scope of the problem that this project aims to resolve, outlining the challenges that will be encountered while working to resolve it. The problem being, delivering medical supplies to rural America in case of another global pandemic.

In rural America, where over 51 million people live, healthcare access is notably limited due to long travel distances to medical facilities and a shortage of healthcare providers. This issue especially became noticeable during the COVID-19 pandemic, where rural communities struggled to access testing services, a crucial factor in controlling viral spread. The lack of effective, accessible testing infrastructure has exposed a critical gap in the healthcare system for these communities, making rapid, reliable testing difficult to implement.

Population-wide testing is costly, and the high rate of false positives reduce its effectiveness. Studies suggest that a focus should be on testing individuals showing symptoms and those in contact with infected persons. However, delivering test kits to these individuals still proves a challenge.

Drones offer a promising solution for delivering test kits to rural areas, helping bridge the last-mile gap where traditional logistics often fall short. Past deployments in countries like Rwanda and Ghana have shown drones' effectiveness in healthcare deliveries to remote regions. However, challenges such as battery limitation and payload capacity must be addressed for drones to serve this role effectively.

By resolving these challenges, drones could play an important role in enhancing the healthcare system, facilitating fast and focused delivery of test kits to rural communities in America, and improving testing accessibility during a future public health crisis.

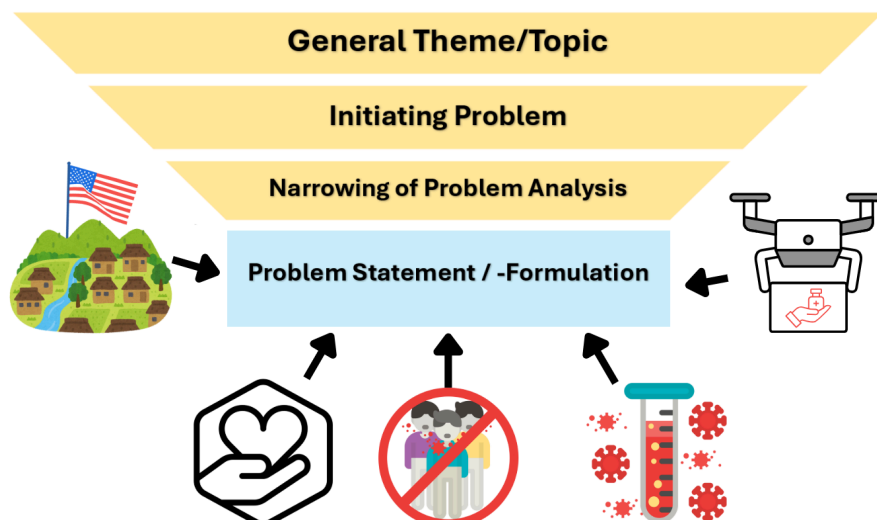


Figure 3: Problem Delimitation

3.2 Problem Statement

With the information from the problem delimitation in Section 3.1, the following problem statement has been formulated:

How can a software solution be developed to minimize the distance of drone routes for delivering test kits to households of individuals exposed to infection during a future pandemic?

4

Methodology & Theory

4.1 Tools & Technologies

When designing a routing program, it is relevant to research existing routing algorithms. These algorithms encompass a variety of approaches, each tailored to address specific routing challenges. Some algorithms are specialized for distinct routing objectives, while others address similar problems, but employ differing methodologies to reach a solution. A relevant routing algorithm is the Branch & Bound algorithm, which offers an effective solution to the routing problem. This algorithm is widely recognized for its applicability in optimizing paths within network and pathfinding contexts.

4.1.1 Programming Language

The programming language used for the solution is C. It offers some advantages compared to other languages. For example, C is often faster and gives low-level access to memory. Additionally, C is widely used, making resources and documentation easily accessible.

4.1.2 IDE

To write code, CLion from JetBrains will be used as IDE. CLion offers cross-platform development testing directly in the IDE and CMake integration.

4.1.3 Version Control

Git with GitHub will be used for version control and for sharing code from multiple developers.

4.2 Graph Data Structure & Algorithms

This project utilizes graphs to determine the optimal path between points. Graphs consist of nodes interconnected by edges, which can be assigned weights to represent traversal costs. In this context, the weight of an edge signifies the traveling distance required to traverse that segment. This approach allows for simulating a drone's paths to find the fastest and most optimal route. The vertices, or nodes, of the graph, will represent delivery points on a map and can be households, institutions, or any other instance in need of medical supplies. The edges will represent the distance between delivery points, and because the solution is utilizing drones, pathways will not be limited to roads but will represent the most direct path between two vertices.

4.2.1 Graph Theory

Graph theory is a branch of mathematics focused on studying graphs. In graph theory, graphs are structures used to model relationships and connections between objects. A graph consists of nodes (also known as vertices) and edges, which are the connections between the nodes.

A graph can be directed or undirected, which is whether or not the edges have a direction, meaning that movement may be restricted like for example a road with a one-way sign. A graph can also be weighted or unweighted. The weight in this context is a value assigned to an edge,

that typically represents the “cost”, distance, or time associated with that particular edge. In an unweighted graph, all edges are treated equally since they don’t have any value associated with them [26].

4.2.2 Graph Representation Using Adjacency Matrices

An adjacency matrix is a way of representing a graph using a matrix, where the elements indicate whether vertices are adjacent. An adjacency matrix is especially useful when working with dense and complete graphs [27]. Figure 4 represents an adjacency matrix for an undirected weighted graph, where “INF” means that there are no edges between the vertices. It is important here to highlight, that this value does not have to be “INF”, it could be 0, INT_MAX or any other random number, it just has to have a value that represents that there are no edges between the vertices.

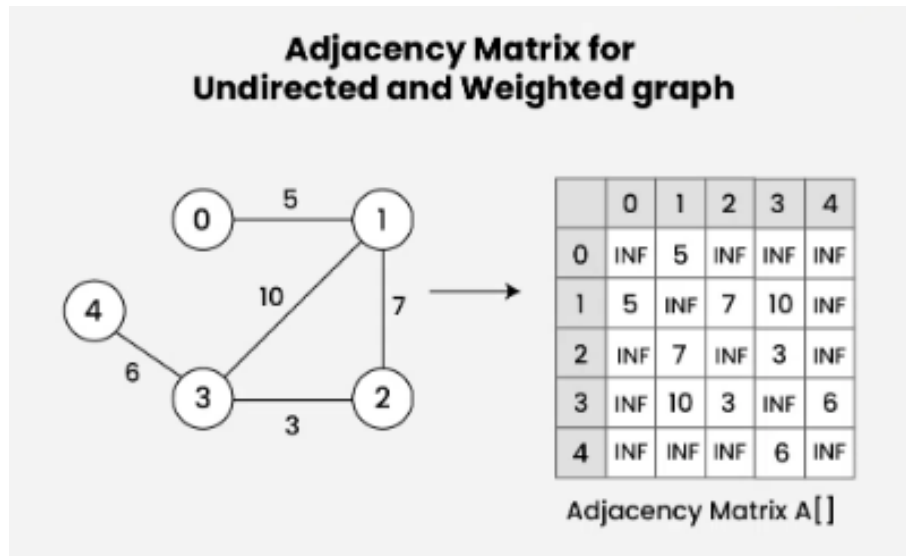


Figure 4: Example adjacency matrix [27].

In a weighted graph, $A[i][j]$ contains the weight of the edge between i and j , or “INF” if no edges exist. For undirected graphs, the adjacency matrix is symmetric because an edge from vertex i to vertex j implies an edge from j to i . The adjacency matrix is useful for determining certain properties of graphs. If every pair of vertices has a path between them the matrix is connected. Some other advantages of adjacency matrices is the simple implementation, and fast edge access where checking whether an edge exists between two vertices is a constant-time operation ($O(1)$) [27].

4.2.2.1 Comparing Adjacency Matrix & Alternative Graph Representations for Complete Graphs

When representing a complete graph, the adjacency matrix provide clear advantages over the other graph representations such as adjacency list and edge lists. Its effectiveness come from the structure of complete graphs, where every node is connected to every other node.

Advantages of an Adjacency Matrix

The adjacency matrix is a two-dimensional $n \cdot n$ array, where n is the number of nodes in the graph. Since all edges are present in a complete graph the adjacency matrix directly maps these connections without needing dynamic data structures like linked lists. The memory usage in an adjacency matrix for a complete graph is therefore always $O(n^2)$, as each of the n nodes has edges to every other node.

Additionally, checking the weight of an edge between two nodes (edge lookup) is an $O(1)$ operation. This constant time access is really beneficial for algorithms requiring repeated edge lookups, such as the Branch and Bound method for solving the Traveling Salesman Problem (TSP). This is also shown in greater detail in Section 5.4.3.1.

At last, the adjacency matrix is in nature compatible with matrix based algorithms and operations, which simplifies the implementation of many graph algorithms, especially those in optimization and combinatorial contexts, where dense complete graphs can occur.

Comparison with Alternative Graph Representations

Adjacency Lists:

For a more detailed explanation of the adjacency list, its memory usage, and the time complexity of the edge lookup, please refer to Section 9.4 in attachments.

While adjacency lists are memory efficient for sparse graphs (a graph with few edges), their advantages disappear for complete graphs. In a complete graph, every node is connected to every other node, meaning each node has $n - 1$ neighbors, where n is the total number of nodes. Storing $n - 1$ neighbors of each node in a complete graph requires $O(n^2)$ space, which matches the memory usage of an adjacency matrix. However, edge lookup operations in the adjacency list require $O(n)$ time, as the list neighbors must be traversed to find a specific edge. This inefficiency limits the effectiveness of adjacency lists for graphs where repeated edge lookups are required.

Edge Lists:

For a more detailed explanation of the edge list, its memory usage, and the time complexity of the edge lookup, please refer to Section 9.5 in attachments.

The memory usage for an edge list in a complete graph is $O(n^2)$, as it stores all the edges directly. This also matches the memory usage of the Adjacency matrix and list. The time complexity for edge lookups in an edge list is $O(n^2)$ in the worst case. This is because the edge list contains all the edges of the graph and thus must scan through the entire list to find a specific edge. In a complete graph, this means that in the worst case, it has to traverse all $O(n^2)$ edges to locate the desired edge. This makes edge lists inefficient for algorithms that require constant time access to edges, such as the Branch and Bound method for TSP.

Conclusion of the Comparison

In conclusion, for complete graphs, the adjacency matrix is superior in cases where repeated edge weight lookups are essential. This is especially important in problems such as TSP, where algorithms like Branch and Bound require repeated access to edge weights. Here the $O(1)$ edge lookup time provided by the adjacency matrix significantly increases computational efficiency and reduces calculation times. Additionally, its simple and natural structure enhances the implementation of graph traversal and optimization algorithms, like Branch and Bound.

This means that, while adjacency lists and edge lists may be more suitable for sparse graphs, the adjacency matrix is the preferred choice for complete graphs, due to its efficient edge storage and edge lookup capabilities. Thus the adjacency matrix will be the desired form of graph representation in the implementation program.

4.2.3 The Traveling Salesman Problem & The Vehicle Routing Problem

The Traveling Salesman Problem (TSP) is a problem, that is concerned with the determination of the shortest path/route, that passes through each of n given points once [28].

The problem is called the Traveling Salesman Problem, because it derives from a scenario, where a salesman wishes to travel by the shortest total distance from his home and to each of the $n - 1$ cities/points, and then back home [29].

The TSP is in mathematical terms all about finding a permutation, $P = \{i_1, i_2, \dots, i_n\}$ of the integers from i_1 to n , where the goal is to minimize the quantity of:

$$a_{i_1 i_2} + a_{i_2 i_3} + \dots + a_{i_n i_1}$$

Where $a_{\alpha\beta}$ is the given distance between each pair of cities from a set of real numbers ($\mathbb{R}$). In more detail, there are only $(n - 1)!$ possible ways to arrange the cities (permutations), which means that TSP becomes computationally expensive to solve when the number of cities increases [29]. Assuming that each pair of cities/points is connected by a path/edge, the total number of different routes through n cities/points can be found by:

$$\frac{1}{2} \cdot n!$$

Even for small numbers of n , the total number of routes is extremely large. e.g. for $n = 15$, there are 653.837.184.000 different routes, because $\frac{1}{2} \cdot n! = \frac{1}{2} \cdot 15! = 653837184000$ [28].

The Vehicle Routing Problem (VRP) is according to [30], one of the most studied optimization problems when it comes to the concern with the optimal design of routes that are used by a group of vehicles, deliver or serve customers or retailers.

So, in short, VRP is a set of identical vehicles, which are located at a central depot. These vehicles are set to be optimally routed for supply customers [30].

There are different variants of this problem, such as Capacitated VRP (CVRP) and VRP with Time Windows (VRPTW). The CVRP contains a set of identical vehicles, that are all usable, and the only constraint is the capacity of the vehicles. And, the VRPTW is where the customers may be served within a defined time interval and therefore are the schedules/routes of the vehicles determined accordingly [30].

4.2.4 The difference between TSP & VRP

This section highlights the differences between TSP and VRP, from the prior knowledge of TSP and VRP.

VRP extends the TSP by considering multiple vehicles and adding real-world constraints, like capacity, time windows, and different start/end points.

Aspect	TSP	VRP
<i>Number of Vehicles</i>	One	Multiple
<i>Objective</i>	Minimize the total distance (or time) for one route	Minimize the total cost for all vehicles across multiple routes
<i>Constraints</i>	No capacity or time window constraints	Can involve vehicle capacity, time windows and multiple depots
<i>Problem Size</i>	Relatively simpler	More complex due to the introduction of multiple vehicles and constraints

Table 1: TSP vs VRP

4.2.5 Euclidean Distance & Graph Creation

What is the Euclidean Distance?

The Euclidean distance is the length of the straight-line between two points/nodes in a space, and it is obtained via the Pythagorean theorem. The definition of the Euclidean distance in a 2D space, states that the shortest possible distance between two points in a 2D space, $A(x_1, y_1)$ and $B(x_2, y_2)$, also known as the Euclidean distance d , is given by the formula:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

How the Euclidean Distance is Used in TSP

In the context of TSP, the delivery points/nodes are represented as coordinates in a 2D plane. The Euclidean distance is used to calculate the distance between every possible pair of nodes, thus forming a complete graph.

A description of how to implement this in a program can be seen in Section 5.4.2. However to put it shortly:

1. The program reads a list of coordinates representing the delivery points (e.g. (x_1, y_1) , (x_2, y_2) , ...)
2. Then, it uses the Euclidean distance formula, to calculate the distance between every pair of points/nodes.
3. At last, the calculated distances are stored in an adjacency matrix. Each cell (i, j) in this matrix represents the distance between node i and node j .

Constructing the Adjacency Matrix:

In the context of the program, an adjacency matrix is a 2D array, where rows and columns represent the nodes and the value at position (i, j) represents the distance between node i and node j . This also means that if there are n nodes, the adjacency matrix will be an $n \cdot n$ matrix. For a complete graph, all cells (i, j) are filled with the calculated distance, except the diagonal (i, i) , which is set to infinity (∞) because a node does not have a distance to itself.

Mathematical Implementation Example:

Here is an example of how the adjacency matrix is constructed:

Assume the following delivery points/nodes have been read or initialized:

Node 0: (1, 1), Node 1: (4, 5), Node 2: (9, 6)

The next step is to calculate the Euclidean distances between every possible pair of nodes using the Euclidean distance formula:

$$d(0, 1) = \sqrt{(4 - 1)^2 + (5 - 1)^2} = \sqrt{3^2 + 4^2} = 5$$

$$d(0, 2) = \sqrt{(9 - 1)^2 + (6 - 1)^2} = \sqrt{8^2 + 5^2} = \sqrt{64 + 25} = 9.43$$

$$d(1, 2) = \sqrt{(9 - 4)^2 + (6 - 5)^2} = \sqrt{5^2 + 1^2} = \sqrt{25 + 1} = 5.1$$

Hence, the adjacency matrix will in this scenario look like this:

	0	1	2
0	∞	5	9.43
1	5	∞	5.1
2	9.43	5.1	∞

Figure 5: Adjacency matrix filled with Euclidean distances between Nodes

4.2.6 Branch & Bound Method

Branch and Bound is a method used in combinatorial optimization problems, such as TSP, to systematically search for the best solution, in this case, the fastest route between all delivery points. The algorithm works by dividing the problem into smaller subproblems (branches), and then excluding specific branches based on bounds set by the optimal solution. This process repeats until the best solution is found or all branches have been explored/visited. The Branch and Bound algorithm is therefore really useful when solving problems like the traveling salesman problem [31].

Characteristics of the Branch & Bound algorithm:

- **Optimal Solution:** The algorithm is designed to find the optimal solution to an optimization problem, by searching for the solution systematically.
- **Upper & Lower Bounds:** The algorithm uses upper and lower bounds to minimize the search space/interval and exclude subproblems, that can't contain the optimal solution.
- **Pruning/Exclusion:** The algorithm excludes the search “tree” by excluding subproblems that can't contain the optimal solution, or are not worth exploring further.
- **Backtracking:** The algorithm uses backtracking to return to a previous node/vertex in the search “tree” when it sees that a dead end has been reached or that a better solution has been found.

4.2.6.1 Traveling Salesman Problem using Branch & Bound

Given a set of delivery points and the distances between every pair of delivery points, also called a complete graph. The problem is finding the shortest possible route that every delivery point is explored at most once and returns to the starting point [31]. These paths are also called Hamiltonian cycles.

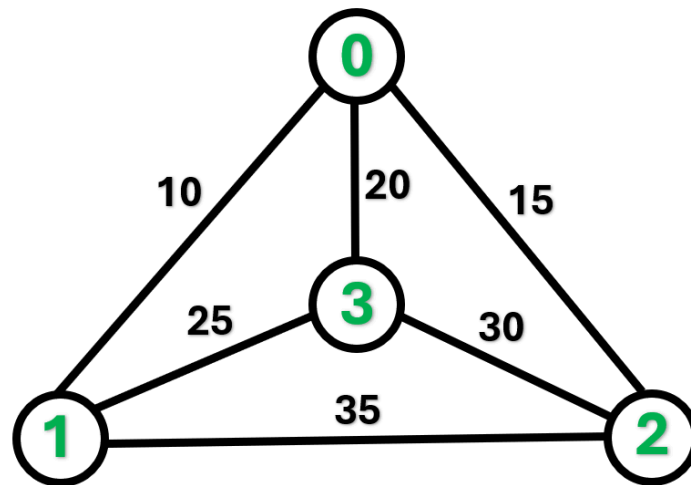


Figure 6: Example of a Travelling Salesman Problem Route [31].

In Figure 6 there is an graph example, where the best solution (shortest solution = minimal cost) for the TSP-route is: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$. The cost (distance in this case) of the fastest route is: $10 + 25 + 30 + 15 = 80$.

For further clarification, check Section 5.4.3 for a comprehensive and detailed explanation of the Branch & Bound algorithm applied on a graph.

5.1 Solution Constraints & Limitations

Some abstractions must be made for the solution to align with this project's scope. Any abstractions will worsen the solution's relation to the real world and therefore include certain constraints and limitations. These limitations will be discussed in the section below

5.1.1 TSP or VRP

The decision to focus on the Traveling Salesman Problem (TSP) rather than the Vehicle Routing Problem (VRP) is due to the project's scope and time constraints. TSP is more aligned with the project's objectives, as it focuses on finding the shortest route that visits each delivery point once and returns to the start. In contrast, VRP involves multiple vehicles and additional complexities, making it too time-consuming and beyond this project's scope.

5.1.2 Input Size & Complexity

When an individual becomes infected, they may have had close contact with people across multiple cities. This solution focuses solely on finding an optimal path between households close to each other, abstracting away the problem of locating households and grouping them based on their locations. The input for the algorithm will be a list of x and y coordinates, representing the locations of households within a group. This approach operates on a simplified 2D plane, meaning the solution and the drones do not need to account for obstacles such as hills, buildings, or trees when planning the route. As a result, Euclidean distances can be used to calculate the lengths between delivery points efficiently.

5.1.3 Effect from Wind & Weather

Another limitation will be made when abstracting for weather conditions. The elements of nature like powerful winds or rain will impact the distance drones can cover in a specific time. A weighted graph between households will have to have its weights reevaluated based on the direction of the wind, making upwind paths weigh higher because of the extra travel time. This is not a problem the proposed solution in this report will tackle. The result is a symmetric graph, where the cost to travel between node i and node j is the same in both directions.

5.1.4 Battery, Charging & Weight Limitations

Lastly, the solution program will not consider the size of the battery, the battery life, or the payload capacity of the drones used in the implementation. In the implementation, drones will have enough battery capacity to cover any path between the input list of households and can carry all deliveries at the same time.

5.2 Product Requirements

Specific requirements must be established to create a solution for test kit delivery. Reviewing these requirements throughout the development process ensures that the problem is being solved. In this project, three different methods for setting up requirements will be used: functional requirements, non-functional requirements, and MoSCoW.

5.2.1 Functional & Non-functional requirements

Functional requirements describe the specific features a solution must include, and what tasks it needs to solve. These drive the core design and functionality of the program. In most cases, the user does not care about how a problem is handled in the program, but more about them getting the outcome they desire. This is where functional requirements are useful. Non-functional requirements focus on the performance of the system. These affect the system's architecture and what algorithms are used to solve the problem. [32]

5.2.1.1 Functional Requirements

The functional requirements can be seen below in Table 2. It outlines the essential functional requirements and description for the desired program, in C, designed to compute the shortest path between delivery points:

Functional requirements	Description
Accept file as input	The program has to be able to read a text file containing the “coordinates” of the delivery points.
Distance between delivery points	The program need to calculate the euclidean distances between delivery points.
Find the shortest path between delivery points	The program must find the shortest path from a start point and then between all the delivery points, and then return to the start point. The algorithm, Branch & Bound, have to be used to find the shortest path.
Output result to user	The program must result in a output that can be read by a user. With the output being the shortest path and the “length cost” of the shortest path.

Table 2: Functional requirements

5.2.1.2 Non-functional Requirements

The non-functional requirements can be seen below in Table 3, where it lists the key non-functional requirements and their descriptions for the program. These requirements focus on the program’s accuracy and usability.

Non-functional requirements	Description
Accuracy	The program should always find the most optimal route
Usability	It should be easy for a user to change the input for the program

Table 3: Non-functional requirements

5.2.2 MoSCoW

The MoSCoW method is used to set up requirements for a product in the form of “must have”, “should have”, “could have”, and “will not have”. Must-have requirements are essential for the product to solve the solution. these are the base minimum requirements. Should-have requirements are not essential but increase the end-user value. Could-have requirements could be nice to have, but if the product is behind schedule, these will be cut first. The Will-not-haves are features that could be implemented in later iterations of the product [33]. These will most likely fall out of the scope of this project.

Must have	Should have	Could have	Will not have
Find optimal path	Read input from file	Output visualized route on graph	Include charging stations to extend the route
Accept variable input	Assign number of nodes based on the number of nodes in the input file	Loading bar for longer run times	
Find distance between delivery points			
Output result to user			

Table 4: MoSCoW requirement table

These requirements give a clear framework for working iteratively with the program and makes it easy to rank features based on importance. First, the must-have requirements will be filled, then the should-have, and if the schedule allows for it, the could-features can be implemented.

5.3 Program Structure

This section will cover the overall structure of the program. Giving an idea of how the different parts of the solution fit together.

5.3.1 High-level Flowchart

The flowchart in Figure 7 describes the program's main functionalities and how it will run. First, the program gets a comma-separated value file (.CSV) as input and opens it. The file contains a list of household locations represented by an x , and a y value. The program then creates a node representing each household and an edge connecting each node to all other nodes. A weight is given to each edge based on the distance between the nodes it connects. One node is initialized as the starting node, from which all routes must start and end. From here a pathfinding algorithm is executed to find the shortest route, that visits all nodes and returns to the starting node. This algorithm is further explained in Section 5.4.3. The shortest route is then returned to the user.

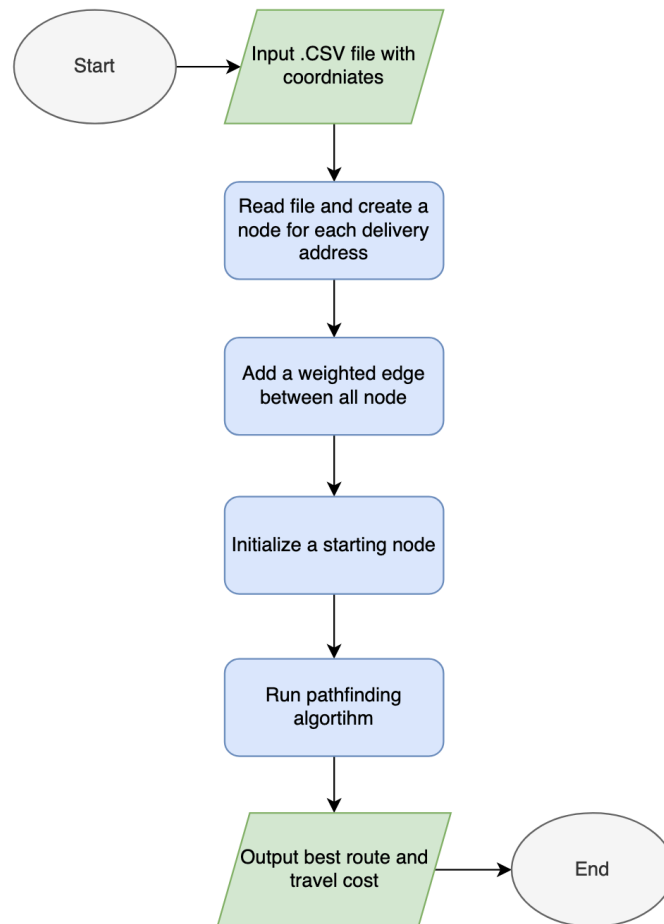


Figure 7: Program structure flowchart

5.4 Algorithm design

This section is a walk-through of how the different functionalities and algorithms implemented in the program solution work.

5.4.1 Importing Graph From a File

The first step in the program is to read household addresses from a .CSV file. These addresses are represented by x and y coordinates, which will be used to calculate the distance between each node. A .CSV or comma-separated value file, can be read like a table, making it easy to work with and scalable because more rows can be added if the number of households increases. The structure of the .CSV file is shown as an example in Figure 8. Here are 2 columns, separated by a comma, and 8 rows, separated by a new line. For simplicity, the coordinates are represented as integer values. The program can read both integer and floating-point values. The first row is a header and is unnecessary for the program's functionality, but it gives anyone reading the code an understanding of the values.

1	x-coordinate, y-coordinate
2	4,8
3	6,1
4	6,10
5	7,2
6	5,9
7	0,6
8	8,10

Figure 8: Example of .CSV file structure

To save the coordinate sets together, a structure called Node is defined to hold an x and a y value. An array of size equal to the number of coordinate sets stores these nodes, which for now holds no values. The coordinate sets are read into each node in the array using the following logic, skipping the first row, as this is a header.

```
FOR Each row in the file
    Read  $x$  and  $y$  value from row
    Current Node  $x, y := x, y$  pair from row
IF More rows
    Jump to next row
ELSE
    Return Nodes array
End FOR
```

Now all nodes have been assigned x and y values. To calculate the edge weights from each node to every other node, a new function is called. See Section 5.4.2.

5.4.2 Edge Weight Algorithm

The weight of an edge between two nodes is equal to the distance a drone has to travel to get from one node to the other. As drones fly in a straight path, that length is the Euclidian distance between the two nodes. An adjacency matrix with *width* and *height* equal to the number of nodes, store edge weights. See an example of a matrix in Section 4.2.2

The formula for Euclidian distance $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ is used to fill out each cell of the matrix by calculating the distance between each node's coordinate set (x, y) . To optimize the program, the distance from one node to itself is set to zero without any calculations, and when the distance from node i to node j is calculated it is also set as the distance from node j to node i simultaneously.

```
FOR Each possible Node pair  $[i, j]$ 
  IF Node  $i ==$  Node  $j$ 
    Set distance = 0
  ELSE
    matrix $[i][j]$  = Euclidian( $i, j$ )
    matrix $[j][i]$  = matrix $[i][j]$ 
  Return adjacency matrix
End FOR
```

The result is a filled-out adjacency matrix like the one seen in Figure 9, where the distance between two given nodes is directly accessible in the matrix. The entire left-to-right diagonal of the matrix has an edge weight of 0, this is because these are the distances from a node to itself. Additionally, the matrix is mirrored across this diagonal, because the distance from node i to node j is the same as that from node j to node i .

	a	b	c	d	e
a	0	4	12	7	3
b	4	0	5	1	2
c	12	5	0	11	4
d	7	1	11	0	3
e	3	2	4	3	0

Figure 9: Example adjacency matrix

The next step is to find the shortest path visiting all nodes on the graph represented by the resulting matrix. This is done using a branch and bound method, explained in Section 5.4.3.

5.4.3 Branch & Bound Algorithm

This section will focus on the pseudocode for the Branch & Bound algorithm:

Function TSP(matrix):

Initialize final_result to a large value (e.g. infinity or INT_MAX)

Initialize final_route to store the best route

Initialize visited array to mark all nodes as unvisited

FOR Each node:

 Add the smallest and second-smallest outgoing edge costs

End FOR

Divide the total sum by 2 to get the initial bound

Mark the starting node (e.g. Node 0) as visited

Initialize current_route with the starting node (Node 0)

Call TSP_Recursion(matrix, current_bound, current_weight, level, current_route)

Return final_result and final_route

Function

TSP_Recursion(matrix, current_bound, current_weight, level, current_route):

IF all nodes are visited (level == total number of nodes):

IF a return edge to the starting node exists:

 Calculate the total cost of the current route

IF this cost is less than final_result:

 Update final_result

 Save the current_route to final_route

Return

FOR Each node i :

IF Node i is unvisited and there is an edge from the current node to i :

 Save the current state of current_bound and current_weight

 Update current_weight with the cost to travel to Node i

 Adjust current_bound based on the minimum outgoing edge costs

IF current_bound + current_weight < final_result:

 Add Node i to current_route

 Mark Node i as visited

 Recursively call TSP_Recursion for the next level

ELSE

 Restore the previous state (backtracking)

 Mark Node i as unvisited

Function Calculate_Bound(matrix, current_node, next_node, level):

IF level == 1:

 Subtract the smallest outgoing edge cost of current_node and next_node

ELSE

 Subtract the second-smallest outgoing edge cost of current_node and smallest of next_node

Return the updated bound

Function Find_Minimum_Edges(matrix, node):

 Find and return the smallest and second-smallest outgoing edge costs for the given node

Explanation of Pseudocode

Initialization:

The algorithm begins by setting final_result to a large value and initializing final_route to store the best route found. The visited array tracks which nodes have been visited and the initial bound is calculated by adding the smallest and second-smallest outgoing edge costs for each node, divided by 2. The algorithm starts at Node 0 and calls the recursive function TSP_Recursion with the initial bound, weight, and route.

Recursive Exploration:

In TSP_Recursion, the algorithm checks if all nodes have been visited. If so, it calculates the total cost and updates final_result and final_route if this route is better. If not, all nodes are visited and the algorithm iterates through unvisited reachable nodes. For each reachable node, the algorithm saves the current state, updates the cost, and adjusts the bound. If the sum of the bound and the current weight is less than final_result, the node is added to the route, marked as visited, and the recursion proceeds. After exploring a node the algorithm backtracks by restoring the previous state to explore alternative routes.

Bound Calculation:

Two distinct edges must be traversed when visiting a node in a Hamiltonian cycle: one edge must lead into the node, and another must lead out. In an optimal Hamiltonian cycle, the total cost is minimized by selecting the cheapest possible edges.

For any given node, the minimum cost contribution is determined by its two smallest outgoing edge costs—the first minimum ($C1$) and the second minimum ($C2$). These two edges represent the least expensive way to include the node in a cycle.

However, since each edge is shared between two nodes in the cycle, directly summing the first and second minimum costs for all nodes would result in double-counting the shared edges. To correct for this, the average contribution of the first and second minimum edges is used instead. This ensures that the calculation reflects the true lower bound on the cost of a Hamiltonian cycle without overestimating.

Backtracking:

After exploring each node, the algorithm restores the previous state, ensuring that alternative paths are considered and explored.

Result:

Once the recursion ends, the best route final_route and its cost final_result are returned as the solution.

5.4.3.1 Example Breakdown of the Branch & Bound algorithm on TSP on a Graph:

Below, in Figure 10, there is an example consisting of a graph, which contains nodes and edges between nodes. This section provides a step-by-step walk-through of solving the Traveling Salesman Problem on this graph using the Branch and Bound algorithm.

Notice that the graph, in Figure 10, is not complete, meaning that not every pair of distinct nodes/vertices are connected by an edge. In the implementation section of the report, there will be used a complete graph, however, to maintain simplicity in this example, an incomplete graph is used. This does not change the process by which the Branch & Bound algorithm approaches the graph, but it simplifies the example, due to there not being a large number of edges between nodes, as there would be in a complete graph.

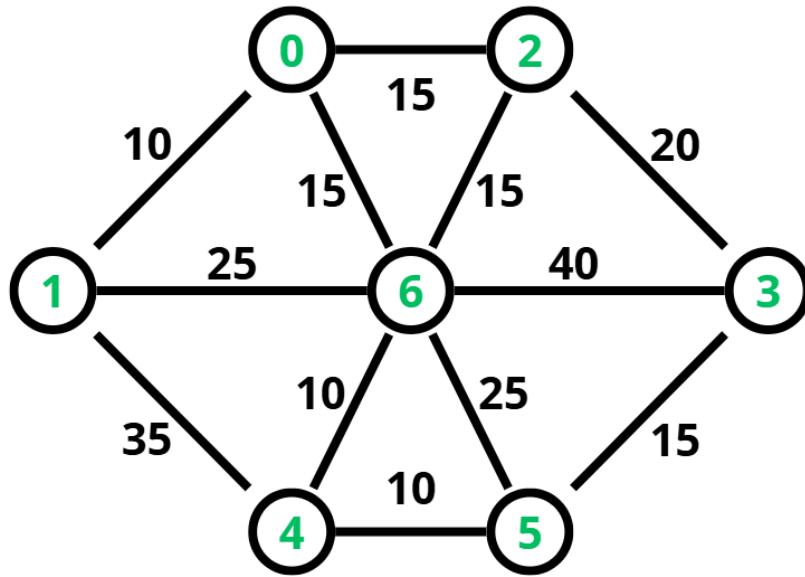


Figure 10: Weighted Graph

Graph Details:

These are the following graph details, which are known:

- Nodes: {0, 1, 2, 3, 4, 5, 6}

The weight of the edges can be seen in Table 4 below:

	0	1	2	3	4	5	6
0	INF	10	15	INF	INF	INF	15
1	10	INF	INF	INF	35	INF	25
2	15	INF	INF	20	INF	INF	15
3	INF	INF	20	INF	INF	15	40
4	INF	35	INF	INF	INF	10	10
5	INF	INF	INF	15	10	INF	25
6	15	25	15	40	10	25	INF

Table 4: Adjacency Matrix, containing distances/weights of the edges between nodes

The weights of edges that doesn't exist are set to a large value, like INF (infinity). This is to ensure that these edges will never be chosen for the route by the Branch & Bound algorithm.

Goal:

The goal is to find the shortest route visiting all nodes exactly once and returning to the starting node.

5.4.3.1.1 Initial Setup - Step 1:

The algorithm begins by calculating an initial lower bound using the adjacency matrix. This lower bound provides an estimate of the minimum cost required to traverse the graph while ensuring that all nodes are visited.

Initial Lower Bound Calculation:

The lower bound is found as the sum of the averages of the first and second minimum edge cost from each node.

Node Contributions:

A contribution for a node can be found with the following formula:

$$\text{Contribution for a node } x = \frac{\text{Minimum edge weight} + \text{Second Minimum edge weight}}{2}$$

- **Node 0:**
 - Minimum edge weight = 10 (to Node 1)
 - Second minimum edge weight = 15 (to Node 2 or Node 6)
 - **Contribution:** $\frac{10+15}{2} = 12.5$
- **Node 1:**
 - Minimum edge weight = 10 (to Node 0)
 - Second minimum edge weight = 25 (to Node 6)
 - **Contribution:** $\frac{10+25}{2} = 17.5$
- **Node 2:**
 - Minimum edge weight = 15 (to Node 0 or Node 6)
 - Second minimum edge weight = 15 (to Node 0 or Node 6)
 - **Contribution:** $\frac{15+15}{2} = 15.0$
- **Node 3:**
 - Minimum edge weight = 15 (to Node 5)
 - Second minimum edge weight = 20 (to Node 2)
 - **Contribution:** $\frac{15+20}{2} = 17.5$
- **Node 4:**
 - Minimum edge weight = 10 (to Node 5 or Node 6)
 - Second minimum edge weight = 10 (to Node 5 or Node 6)
 - **Contribution:** $\frac{10+10}{2} = 10.0$
- **Node 5:**
 - Minimum edge weight = 10 (to Node 4)
 - Second minimum edge weight = 15 (to Node 3)
 - **Contribution:** $\frac{10+15}{2} = 12.5$
- **Node 6:**
 - Minimum edge weight = 10 (to Node 4)
 - Second minimum edge weight = 15 (to Node 0 or Node 2)
 - **Contribution:** $\frac{10+15}{2} = 12.5$

Initial Lower Bound:

So, the initial lower bound is:

$$LB = 12.5 + 17.5 + 15.0 + 17.5 + 10.0 + 12.5 + 12.5 = 97.5$$

5.4.3.1.2 Start Recursive Exploration - Step 2

Once the initial lower bound (LB) is calculated, the algorithm begins the recursive exploration of possible routes, starting from node 0. At each step, the algorithm evaluates potential paths based on the edge costs in the adjacency matrix and whether they can lead to a solution better than the current best-known result (final_result)

Preparation for Recursive Exploration:

- The current weight (current_weight) is initialized to 0, given that no edges have been traversed yet.
- The initial lower bound (LB) is calculated as 97.5 using the method described earlier in Step 1.
- Node 0 is marked as the starting point.
- Setting final_result to ∞ . By initializing final_result to ∞ , any valid solution found during the traversal will be smaller than this initial value, ensuring the first complete tour is stored as the current best solution

Possible Paths from node 0:

From node 0, all possible paths/steps must be evaluated based on the adjacency matrix:

	0	1	2	3	4	5	6
0	INF	10	15	INF	INF	INF	15
1	10	INF	INF	INF	35	INF	25
2	15	INF	INF	20	INF	INF	15
3	INF	INF	20	INF	INF	15	40
4	INF	35	INF	INF	INF	10	10
5	INF	INF	INF	15	10	INF	25
6	15	25	15	40	10	25	INF

Figure 11: Adjacency Matrix, containing distances/weights of the edges between nodes

Figure 11 shows 3 possible paths/steps from node 0 to another node. The paths are the following:

- $0 \rightarrow 1$: (Cost = 10)
- $0 \rightarrow 2$: (Cost = 15)
- $0 \rightarrow 6$: (Cost = 15)

For each route/path, the algorithm performs the following steps:

5.4.3.1.3 Exploring Paths from Node 0:

Path 0 → 2:

Update Current Weight: Add the cost of traveling from Node 0 to Node 2:

$$\text{current_weight}_{\text{updated}} = \text{current_weight} + \text{Travel Cost from Node A to B} = 0 + 15 = 15$$

Update/Recalculate Lower Bound: The lower bound is adjusted by subtracting the average of the smallest outgoing edge of Node 0 (10) and the smallest edge of Node 2 (15) from the initial bound.

$$\text{LB}_{\text{updated}} = \text{LB} - \frac{\text{Smallest edge cost of Node A} + \text{Smallest edge cost of Node B}}{2}$$

Calculated with values:

$$\text{LB}_{\text{updated}} = 97.5 - \frac{10 + 15}{2} = 97.5 - 12.5 = 85$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (initially ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 15 + 85 = 100$$

Since $100 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 2.

Recursive Call: Mark node 2 as visited and recursively evaluate the remaining nodes from this point.

Exploring Paths from Node 2:

Path 2 → 3:

Update Current Weight: Add the cost of traveling from Node 2 to Node 3:

$$\text{current_weight} = 15 + 20 = 35$$

Update/Recalculate Lower Bound: Subtract the average of the smallest outgoing edge of Node 2 (15) and the smallest edge of Node 3 (15) from the lower bound.

$$\text{LB}_{\text{updated}} = 85 - \frac{15 + 15}{2} = 85 - 15 = 70$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (still ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 35 + 70 = 105$$

Since $105 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 3.

Recursive Call: Mark node 3 as visited and recursively evaluate the remaining nodes from this point.

Want the rest of the example?

The rest of the recursive exploration example breakdown of the Branch & Bound algorithm on TSP on a Graph can be found in Section 9.1 under attachments.

5.4.3.2 Processes of the B&B algorithm on TSP with attention to the Example

The Result of the Example & Final Result Update:

In Section 9.1 and partially in Section 5.4.3.1 a breakdown of the Branch & Bound algorithm is shown on an example Graph. In the breakdown example it only showed one complete route/tour, which has the following route/path:

$$0 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 1 \rightarrow 0$$

But, because the final_result is set to ∞ the first exploration of paths will always result in a complete path because all possible complete routes always have a total cost, which is less than ∞ . This was to ensure that the first complete route's total cost, would become the new value of final result.

What happens when exploring other possible routes?

As the algorithm explores valid routes and calculates their potential total costs, the following happens:

1. If a complete route is found:

The cost of this complete route becomes the new value of `final_result`, on the condition that it is lower than the current `final_result`. This ensures that future comparisons with `final_result` reflect the cost of the best complete route found.

2. While exploring incomplete routes

For every partial route the algorithm computes:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}}$$

This value is compared with `final_result`. If the potential total cost is greater than or equal to `final_result`, the path is pruned/excluded. Otherwise, it explores further.

Why Update `final_result`?

Once `final_result` is updated, the algorithm can prune more paths effectively. Any future path with a potential total cost (`current_weight` + `LB`) that exceeds `final_result` will no longer be explored, reducing unnecessary computations.

Why Subtract the Smallest Outgoing Edge of Node *A* and Node *B*?

When the algorithm decides to visit a node (e.g. going from node *A* to node *B*), the current total cost changes. Specifically:

Node *A*: Once the algorithm chooses to visit *B* from node *A*, node *A*'s outgoing edge is irrelevant (as it cannot revisit *A*). Thus, the smallest outgoing edge cost of *A* is subtracted to account for the least additional cost that visiting *A* could have added to the path.

Node *B*: Since *B* is now the next node, the algorithm will need to visit another node from *B*. Subtracting the smallest outgoing edge of *B* reduces the remaining cost by accounting for the minimum possible cost of leaving *B*.

Together, subtracting the average of these values updates the lower bound in a way that reflects the decision to move from *A* to *B*, ensuring that the cost remains consistent with this decision.

By subtracting the average of the smallest outgoing edge cost of *A* and *B*, the algorithm will be forced to follow the rule that a valid path must account for moving out of *A* and *B*. It also makes it so that the lower bound now includes a more realistic estimate of the minimum additional cost required to complete the route, given the decisions to travel from *A* and *B*. This will make the pruning process more accurate by ensuring that paths with high costs are eliminated early.

How does the algorithm prevent itself from exploring the same routes?

Branching Mechanism at Each Level:

At each level of recursion, the algorithm explores all possible branches, from the current node. These branches are generated in a systematic order. In the example conducted in Section 5.4.3.1, at Node 0, the algorithm sees three possible routes, $0 \rightarrow 1$, $0 \rightarrow 2$ and $0 \rightarrow 6$. The algorithm explores these branches one by one in the order they are defined (e.g. numerical, order of nodes, etc.). Once a branch is fully explored (e.g. all paths starting with $0 \rightarrow 2$), the algorithm marks it as complete and backtracks to the previous recursion level.

Tracking Visited Nodes:

To ensure the algorithm does not repeat the same branch, it keeps track of the visited nodes and when backtracking to a previous level, it remembers which branches have already been explored and skips them. For example at node 0, it first explores $0 \rightarrow 1$. After exploring all paths starting from $0 \rightarrow 1$, it backtracks to node 0, where it will now remember that $0 \rightarrow 1$ has been explored and skips it. The algorithm then moves to the next branch $0 \rightarrow 2$, and so on. This mechanism ensures that each branch is explored exactly once.

Progress Tracking:

The algorithm uses a mechanism (e.g. an iteration counter) to ensure it moves to the next unexplored branch when it backtracks. In short, even though the list of visited nodes gets restored to the previous recursion level, resulting in the state “reset” via backtracking, the algorithm still ensures that it does not explore the same route again via for example an iteration counter.

How does the algorithm handle the starting node (Node 0) in the Recursive Calls?

In the breakdown example in Section 5.4.3.1, the starting node is Node 0. This means that the algorithm begins at Node 0, marking it as visited at the start of the route. However, Node 0 is revisited at the end of a route as desired. The algorithm ensures that after visiting all other nodes, the route concludes by returning to Node 0. It does this by checking the following condition at each step: Have all the other nodes been visited? If yes the algorithm calculates the cost to return to Node 0 and adds it to the total cost of the current route.

Why is the lower bound important and how does it work?

The lower bound helps eliminate paths that are unlikely to lead to optimal solutions, by providing a minimum possible cost estimate from the current state. This ensures that the algorithm is efficient and it minimizes the amount of computational power and time needed to find optimal solutions.

Recursive Exploration

With the knowledge of the recursive exploration process in the B&B algorithm, the approach can be generally defined in the following steps:

For each part path that passes the pruning check (e.g. path $0 \rightarrow 2$), the algorithm proceeds recursively by:

1. **Choose Next Node:**

From the current node, the algorithm evaluates all unvisited nodes. For each valid path, the cost is updated, and the bound is adjusted accordingly.

2. **Check Potential:**

If $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result , the algorithm backtracks to explore alternative paths.

3. **Backtracking:**

After evaluating all possible paths from a node, the state (e.g. current_weight , LB , visited nodes) is restored to its previous state before moving to the next path.

4. **Terminate When All Nodes Are Visited:**

If all nodes are visited ($\text{level} = N$) and the route returns to the starting point, the total cost is calculated. If this cost is less than final_result , the result is updated.

The Optimal Path(s)

In the example conducted in Section 9.1, the following complete route was found:

$$0 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 1 \rightarrow 0$$

This was “unintentionally” also the minimal route, that could be made from the graph via the B&B algorithm. However, TSP solutions are circular, meaning a route can be traversed in two directions. This means there is another best route, which is just the best route but in the reverse direction. In this case, it means that there are the following two solutions:

$$0 \rightarrow 1 \rightarrow 6 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 0 \quad \& \quad 0 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 1 \rightarrow 0$$

Both solutions are equally beneficial, but if there is a scenario where only one solution is allowed, it can be fixed by a conditional statement, which says to disregard reversed optimal paths. The optimal paths from the example can be seen in Figure 12.

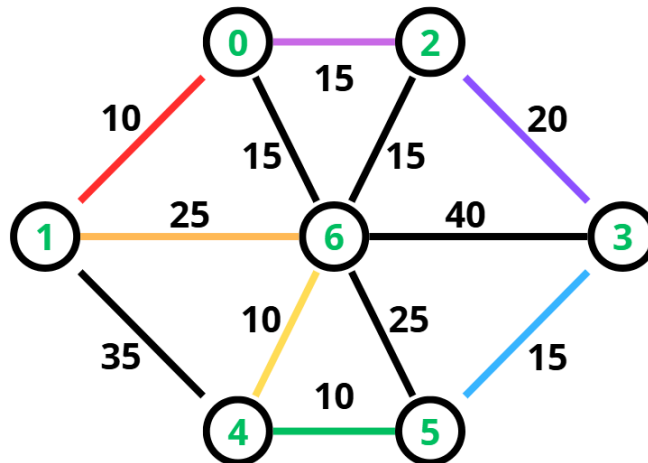


Figure 12: The Optimal Path

5.5 Implementation

5.5.1 Node struct and Node array

Structures in C are user-defined data types useful for storing similar data across multiple entities. In *Node_struct*, each Node stores the same data: an *x*-value and an *y*-value. both as the datatype *double*. Using the keyword *typedef* to define a structure enhances readability by naming this datatype “Node,” allowing the Node datatype to be referenced without using the *struct* keyword.

```
4      typedef struct {
5          double x;
6          double y;
7      } Node;
```

Figure 13: Node struct definition

An array is declared with enough space allocated to store all nodes. The value of “node_count” represents the number of nodes in the given input file. “node_count” is dynamically assigned each time the program is run using the `get_node_count()` function in Figure 15

```
44      const int node_count = get_node_count(filepath);
45
46      Node nodes_array[node_count];
47
```

Figure 14: Node array declaration

To get the node count for the current input file, first, a pointer to the file’s path is declared with the name of *filepointer*. The `fopen()` function takes a filename and a run-mode as its parameters. The filename is given as a formal parameter in the `get_node_count()` function itself, and the run mode is set to “r” for read. Then the legitimacy of the filepointer is checked to prevent errors.

```
12  ↗ int get_node_count(const char *filename) {
13      // Using the implemented FILE struct from C to create a pointer to a files location
14      FILE* filepointer = fopen(filename, mode: "r");
15
16      // If the file is not found, print it to the user and return
17      if (!filepointer) {
18          printf("Can't open file\n");
19          printf("filepath %s", filename);
20          return -1;
21      }
22
```

Figure 15: Get node count function 1/2

Then a counter is initialized to keep track of the number of nodes in the file. The number of nodes is determined from the number of rows in the input file, as each row represents one set of coordinates. The counter is initialized as “`int count = -1`” to disregard the header row. Using a for-loop every character in the file is read until it meets the EOF (end-of-file) indicator. For each line break (“`\n`”), the count is incremented by 1. Lastly, the file is closed, the number of nodes is printed to the user, and the value of *count* is returned and used around the program.

```

23     // Create a counter to return the total number of nodes
24     int count = -1;
25     // Each character in the file is read, until it is equal to the value of "EOF" (end-of-file)
26     for (char c = getc(filepointer); c != EOF; c = getc(filepointer))
27     |     if (c == '\n') // Increment count for each newline in the file
28     |     |     count += 1;
29
30     // Close the file and return count
31     fclose(filepointer);
32     printf("The number of nodes is: %d\n", count); // Print the number of nodes to the user
33     return count;
34 }
35

```

Figure 16: Get node count function 2/2

5.5.2 Reading nodes from file

Coordinates are read from a file to populate the *node_array*. As described in Section 5.4.1 every row of the file is read one by one and the *x* and *y* values are assigned to a node in the array. First, a pointer is created to the file and opened in read mode. Then the program checks for any errors in finding an opening the file in Figure 17

```

40 → void load_nodes_from_file(const char *inputfile, Node nodes_array[], int node_count) {
41     // Using the implemented FILE struct from C to create a pointer to a files location
42     FILE* fp = fopen(filename: inputfile, mode: "r");
43
44     // If the file is not found, print an error to the user, and return.
45     if (!fp) {
46         printf("Error: Filepath not found\n");
47         printf("filename %s", inputfile);
48         return;
49     }

```

Figure 17: Load node from file function 1/3

A variable “buffer” is initialized with space for 100 characters in Figure 18. One line is read using `fgets()`. This is to discard the header row of the input file.

```

51     // Setting a buffer to read each line onto, the buffer is set to hold 100 characters
52     char buffer[100];
53     fgets(buffer, 100, fp); // Read and discard the first line
54

```

Figure 18: Load node from file 2/3

Inside a for-loop in Figure 19, each line in the program is then read into the buffer and the x and y values are scanned from the buffer using `sscanf()`. Two if-statements validate the value in the buffer by checking if the line has been read, and if the values in the line are formatted correctly.

```

56     for (int i = 0; i < node_count; i++) {
57         double x_value = 0.0, y_value = 0.0;
58
59         // Read a line from the file
60         if (!fgets(buffer, sizeof(buffer), fp)) {
61             fprintf(stderr, "Error: Failed to read line %d from input file\n", i + 1);
62             exit(EXIT_FAILURE);
63         }
64         // Scan the coordinates from a line
65         if (sscanf(buffer, "%lf,%lf", &x_value, &y_value) != 2) {
66             fprintf(stderr, "Error: Line %d is not formatted correctly: %s(should be x_value,y_value)", i + 1, buffer);
67             exit(EXIT_FAILURE);
68         }
69         // Round and assign the values to each node
70         nodes_array[i].x = (x_value);
71         nodes_array[i].y = (y_value);
72         // Print the node details
73         printf("Node %d: (%.3lf, %.3lf)\n", i, nodes_array[i].x, nodes_array[i].y);
74     }
75 }

```

Figure 19: Load node from file 3/3

5.5.3 Calculating edge weights

As mentioned in Section 5.4.2 an adjacency matrix stores the weight of edges between nodes. This matrix is initialized with both a *width* and a *height* equal to the number of nodes. Here the value of `node_count` from the `get_node_count()` function is used for initialization.

```

74
75     double matrix[node_count][node_count];
76

```

Figure 20: Matrix declaration and initialization

Logic also described in Section 5.4.2 is used to calculate edge weight, but here is the code implementation. The function `calculate_edges()` takes `node_count`, the edge matrix, and the nodes array as parameters. Two for-loops iterate over the matrix and assign edge weight using coordinate values from the nodes array.

If the matrix is split along its left-right diagonal only one-half of the values has to be calculated, as it can be symmetrically assigned. This is done in the inner for-loop where j is set to i . For each row the function moves down, one less value has to be calculated, as the others are already filled in. After all edge weights are assigned for the graph, the function ends. No value is returned as the assignments happened in the original matrix.

```

19 → void calculate_edges(const int node_count, double matrix[node_count][node_count], Node nodes[]) {
20     //The two nested loops are used to iterate over every possible pair of nodes "(i, j)", where "i" is the
21     //row index and "j" is the column index in the distance matrix:
22     for (int i = 0; i < node_count; i++) {
23         for (int j = i; j < node_count; j++) {
24
25             // If i == j is set to zero as this is distance to self
26             if (i == j) {
27                 matrix[i][j] = 0;
28                 // If j != j calculate edge of weight with Euclidean distance
29             } else {
30                 matrix[i][j] = euclidean_distance(start: nodes[i], goal: nodes[j]);
31                 matrix[j][i] = matrix[i][j]; // Symmetric assignment
32             }
33         }
34     }
35 }

```

Figure 21: Calculate edge weight function

When assigning weight values in the `calculate_edges()` function, another function calculates the Euclidean distance from one node to another. The built-in function `fabs()` is used to get the difference in x and y values from the two nodes as absolute values, assuring no negative difference is used to calculate the distance. The distance from the start node to the end node is then calculated using the Pythagoras theorem.

```

47 → double euclidean_distance(const Node start, const Node goal) {
48     // Using abs() to calculate the absolute difference between 'x' and 'y' coordinates.
49     double dx = fabs(goal.x - start.x);
50     double dy = fabs(goal.y - start.y);
51
52     // Returning the distance between the two nodes.
53     return (sqrt(dx * dx + dy * dy));
54 }

```

Figure 22: Euclidean distance function

5.5.4 Pathfinding using branch and bound method

5.5.4.1 TSP main function

The functionality and logic of the pathfinding algorithm are described in detail in Section 5.4.3. This section will describe the code implementation of this algorithm. The main function here is the `TSP()` function. This is where variables keep track of traversing and state are initialized and where the recursive Branch and Bound functions are called. The first part of the function in Figure 23 is to initialize variables, here arrays are declared to keep track of which routes are visited and the current route. `memset()` is used to fill the arrays with a particular value. All elements in `current_route` are set to -1 to indicate no nodes and all elements visited are set to 0 , as no nodes are revisited yet. At the start of the function “loading ...” is printed to the console to let the user know the program is running.

```

17 void TSP(int size, double matrix[size][size], double* final_result, int final_route[]) {
18     printf("\033[32;1mLoading ... \033[0m \n");
19     fflush(stdout);
20
21     int visited[size]; // Keeps track of visited nodes
22     int current_route[size + 1]; // size + 1 to have space for appending the starting node to end
23     double current_bound = 0; //Initially set current_bound to 0.
24
25     memset(current_route, -1, sizeof(current_route)); // -1 indicates a blank space in the array
26     memset(visited, 0, sizeof(visited)); // 0 = "not visited", 1 = "visited"

```

Figure 23: TSP() function 1/3

In Figure 24 the initial current bound is calculated. A for-loop iterates over each node and sums their minimum and second-minimum outgoing edge weights. Then that sum is divided by 2, to get an estimate of the cost of the minimum route, as described in Section 5.4.3.1

```

30 for (int i = 0; i < size; i++) {
31
32     // Smallest and second-smallest outgoing edge from each node is summed
33     current_bound += (first_minimum(size, matrix, i) + second_minimum(size, matrix, i));
34 }
35
36 current_bound /= 2;
37

```

Figure 24: TSP() function 2/3

Figure 25 shows the end of this function. Here the first node is determined by setting `visited[0]` equal to one, to mark that node as visited. Additionally, the `current_route` is updated to have node 0 as the first element. Then `TSP_recursion()` is called for the first time.

```

39 //The TSP route is initialized to start from node 0:
40 visited[0] = 1; //visited[0] = 1; marks node 0 as visited.
41 current_route[0] = 0; //current_route[0] = 0; places node 0 at the beginning of "current_route[]".
42
43 //Run the TSP_Recursion() function to begin recursion exploration:
44 TSP_Recursion(size, matrix, current_bound, current_weight: 0, level: 1,
45 | current_route, final_result, visited, final_route);
46

```

Figure 25: TSP() function 3/3

5.5.4.2 Recursive function

The base case of the recursive function applies when a path is complete, and all nodes have been visited. This is seen in Figure 26. The condition for the base case evaluates the value of `level`, which is the depth of the current path. If `level` is equal to `size` all nodes have been visited in that given path, ending the recursion. The cost of the current route is saved and compared to the minimum cost so far.

```

115 → void TSP_Recursion(int size, double matrix[size][size], double current_bound, double current_weight,
116     int level, int current_route[], double *final_result, int visited[], int final_route[]) {
117     if (level == size) { // Check is all nodes have been visited
118
119         //Initializing the variable "current_result"
120         double current_result = current_weight + matrix[current_route[level - 1]][current_route[0]];
121
122         //Check if "current_result" is less than "final_result"
123         if (current_result < *final_result) {
124             copy_To_Final(size, current_route, final_route);
125             *final_result = current_result;
126         }
127
128         //After updating "final_result" and "final_route"
129         return;
130     }

```

Figure 26: Recursion 1/4

If the current route cost is smaller, a helper function `copy_to_final()` is called to copy the `current_route` to `final_route` in Figure 27. Here all nodes are copied from `current_route` and the last element in `final_route` is set to the starting node to make a full loop.

```

245 → void copy_To_Final(int size, int current_route[], int final_route[]) {
246
247     // Copy current route to final route
248     for (int i = 0; i < size; i++) {
249         final_route[i] = current_route[i];
250     }
251
252     // Returning to the starting node
253     final_route[size] = current_route[0];
254 }

```

Figure 27: Copy of final function

If all nodes have not been visited, a for-loop in Figure 28 integrates over all nodes and checks if they are connected with the current node, and if the node has not been visited. If so, a variable `temp` stores the `current_bound`, and a new `current_bound` and `current_weight` is calculated to travel to the new node. A new current bound is then calculated for the drone's potential next node. Here two helper functions are used to calculate `first_minimum` and `second_minimum` in Figure 29 and Figure 30. If the drone is at the starting node, when "`level = 1`", `current_bound` is computed using the `first_minimum` of the starting node, and the `first_minimum` from the potential next node. If the drone is deeper in the path, `second_minimum` is used for the current node.

```

176 // It all nodes have not been visited
177 for (int i = 0; i < size; i++) {
178
179     // Consider travel cost to the next not visited node
180     if (matrix[current_route[level - 1]][i] != 0 && visited[i] == 0) {
181
182         // Store the current bound in temp
183         double temp = current_bound;
184
185         // Add the cost of traveling to node[i] to current_weight
186         current_weight += matrix[current_route[level - 1]][i];
187
188         // Subtract average lowest outgoing edge of node[i]
189         current_bound -= (second_minimum(size, matrix, i: current_route[level - 1])
190             + first_minimum(size, matrix, i)) / 2;
191

```

Figure 28: Recursion 2/4

A for-loop iterates over each edge from a given node to find the first minimum distance. To start *first* is set to *MAXFLOAT*, and whenever a lower edge weight is found, *first* is updated to hold that value.

```

60 ↵ int first_minimum(int size, double matrix[size][size], int i) {
61     double first = MAXFLOAT;
62
63     // check outgoing edge weight of node[i]
64     for (int j = 0; j < size; j++) {
65
66         // If new lowest weight is found, update first
67         if (matrix[i][j] < first && i != j) {
68             first = matrix[i][j];
69         }
70     }
71     return first;
72 }

```

Figure 29: First minimum function

The second minimum is found much like the first minimum, but when a distance lower than *first* is found, *second* is set to the current value of *first*, after which *first* is updated with the newly found lesser value. If a value lower than *second* but greater than *first* is found, only *second* is updated. If the value is equal to *first*, the value of *second* is not updated to ensure *first* is always smaller than *second*.

```

85 ↩ int second_minimum(int size, double matrix[size][size], int i) {
86     double first = MAXFLOAT;
87     double second = MAXFLOAT;
88
89     // check outgoing edge weight of node[i]
90     for (int j = 0; j < size; j++) {
91
92         if (i == j) { //Skip the current iteration if "i == j"
93             continue;
94         }
95
96         // If new lowest is found
97         if (matrix[i][j] <= first) {
98             second = first;
99             first = matrix[i][j];
100
101             // If a value between first and second is found, only update second
102         } else if (matrix[i][j] <= second && matrix[i][j] != first) {
103             second = matrix[i][j];
104         }
105     }
106     return second;
107 }

```

Figure 30: Second minimum function

In Figure 31 the function checks if the new branch has to be pruned, by comparing the sum of *current_cound* and *current_weight* with *final_result*. If the current route is still valid *current_route* is updated with this node and the node is added to the visited array. Then the recursive function is called again from that node.

```

156 // If true, prune this branch from the tree
157 if (current_bound + current_weight < *final_result) {
158
159     // Add node[i] to the rute
160     current_route[level] = i;
161     visited[i] = 1;
162
163     // - "TSP_Recursion()": Is called to move to the next level:
164     TSP_Recursion(size, matrix, current_bound, current_weight, level + 1,
165                 current_route, final_result, visited, final_route);
166 }

```

Figure 31: Recursion 3/4

If the new branch is too costly it is pruned by backtracking to the last node. This is done by subtracting the node's weight from *curren_weight* and setting *current_bound* back to its prior value stored in *temp* in Figure 32. Lastly, the node is removed from the visited array.

```

168 // To prune the branch, backtrack one node
169 current_weight -= matrix[current_route[level - 1]][i];
170 current_bound = temp;
171 visited[i] = 0;
172 }
173 }
174 }

```

Figure 32: Recursion 4/4

5.5.5 Output

The output for the program is the minimal cost to travel between each point and back to the start, the path taken, as well as how long it took the program to calculate this route (See Figure 35). The minimal cost is simply printed from the variable *final_result*, the path is printed using a for-loop to iterate over each element in the *final_route* array, and the time for calculation is returned from function *print_execution_time()* Figure 33.

```
98 // Output:
99 printf("Minimum cost: %lf\n", final_result); // Prints the lowest travel cost found.
100
101 // Print the minimum path
102 printf("Path Taken: ");
103 for (int i = 0; i <= node_count; i++) {
104     printf("%d ", final_route[i]);
105 }
106
107 // Print the time it took to calculate
108 printf("\n");
109 print_execution_time(start);
110
111 //Stops the program successfully:
112 return EXIT_SUCCESS;
113 }
```

Figure 33: Output logic

The timer function uses the C-library “timer”. A timer is started before calling the TSP() function and the timer is ended when calling the *print_execution_time()* function in Figure 34. This function calculates the difference in end and start time and prints it to the user in seconds. In the *printf()* statement “%.3f” is used to get the time with 3 decimal points.

```
9 void print_execution_time(clock_t start_time) {
10     clock_t end_time = clock(); // Stop the timer
11     // = end_time - start_time
12     double execution_time = (double)(end_time - start_time) / CLOCKS_PER_SEC;
13     printf("Execution time: %.3f seconds\n", execution_time); // Print timer to user
14 }
```

Figure 34: Timer function

An example output for a randomly generated graph with 20 nodes is seen in Figure 35, the lowest cost to travel to all nodes and back is 79, the path taken is printed in the console, and the execution time is 6.757 seconds. Random graphs are generated with a function described in Section 9.6

```
Minimum cost: 1293.665165
Path Taken: 0 4 13 6 14 11 9 10 2 7 3 12 1 8 5 0
Execution time: 0.360 seconds

Process finished with exit code 0
```

Figure 35: Example output for a graph with 20 nodes

6

Discussion & Future Developments

This section will provide a discussion of the project and its results, along with suggestions for potential improvements and further development, if the project were to extend beyond its current constraints.

6.1 Discussion

The project faced certain limitations. For example, the algorithm uses complete graphs, which disregard real-world factors, such as terrain, obstacles, and weather conditions.

While the software implementation mostly fulfilled all the functional- and non-functional requirements, there are still some things that need to be discussed or improved upon, when it comes to certain scenarios.

6.2 Scalability

One of the main limitations observed in the software implementation is the scalability of the dataset used. The Branch and Bound algorithm, while accurate and reliable for small to medium-size datasets, displays an exponential growth in computational time as the number of delivery nodes increases. As shown in Section 9.7 the number of total route permutations increases with $n!$, where n is the number of nodes. Even though the lower bound in the Branch and Bound algorithm is capable of excluding a lot of the non-optimal route combinations, it still has to compute an exponential amount of route combinations, when the number of nodes increases. This is a known drawback of combinatorial optimization methods, especially when it comes to problems like TSP. This means that if the implementation software should be improved upon in the future, it should account for scenarios with a large number of nodes. To address this issue, an alternative algorithm could be explored. For example, the Ant Colony Optimization (ACO) algorithm, can provide a near-optimal solution with a noticeable reduction in computation times [34]. While this algorithm might give up some accuracy, it can still be applied to real-world applications requiring quick decision-making, especially when scaling to hundreds of delivery points.

With this knowledge, it could be beatifical to integrate a hybrid approach, where the Branch and Bound algorithm is used for small to medium-sized datasets and then the Ant Colony Optimization algorithm for larger datasets. This could provide a balanced solution that combines precision and efficiency.

6.3 Other Applications

The primary objective of the software program is well-defined, however, it also has the properties to solve a variety of other problems. One notable example is the optimization of delivery routes for food delivery, a task for which the software is well-suited. In general, the program is well-suited for tasks involving route optimization for various types of deliveries or any scenario requiring the efficient path-finding of multiple destinations in the shortest possible time.

6.4 Further Developments

The requirements for solving the pathfinding problems have been met. However, an ideal implementation would incorporate additional functionalities to enhance the program's performance or improve the user experience. These could include showing the user a loading bar to reduce the level of uncertainty when the program is running, outputting the result graphically on a map or a graph, and updating the program to be able to expand routes using charging stations or support trucks.

6.4.1 Incomplete graphs

The proposed solution is designed to take a complete graph as input. This simplifies the problem by connecting all nodes. However, real-world scenarios often involve incomplete graphs, where some node pairs have missing connections, due to physical or regulatory constraints.

The algorithm would set the weight on a missing edge to infinity (∞) in the adjacency matrix. This prevents the algorithm from traveling between those nodes. This approach is especially relevant for drone delivery, where direct paths may be unavailable, because of obstacles in the terrain or as said before, regulatory restrictions.

For future developments, it would be a great idea, to modify the program to use incomplete graphs as well, as it enhances its practical usefulness in real-world cases.

6.4.2 Loading bar

As proposed in Section 5.2.2 a loading bar could be implemented in the solution to show progress to the user and to keep the user assured that the program is still running. Implementing a loading bar could be done by first calculating the total number of possible paths the program will consider. Whenever the program reached the last node of a path or pruned a path, the program would increment a counter and calculate a percentage of paths searched. Then a loading bar would be shown to the user, filled out based on the percentage of searched paths out of all the possible paths. A loading bar is not a key requirement and did not contribute to the core functionality, therefore it has not been implemented.

6.4.3 Graphical output

Incorporating a graphical output into the program would significantly enhance the user experience by providing a visual representation of the computed routes. Instead of simply displaying numerical results in the program console, the program could overlay the optimized path on a map or a graph. This would allow users to naturally understand the route and verify its practicality in real-world applications. While the graphical output is not essential for the program's core functionality, it can still be a valuable addition for future developments.

6.4.4 Charging station and truck-supported delivery

In practice, drones require charging, a factor that is not currently accounted for in the software solution. This limitation represents a challenge that must be addressed in real-world scenarios. A viable approach would involve establishing strategically placed charging stations, as outlined in

Section 2.6.1. Furthermore, incorporating truck-supported deliveries could significantly enhance the efficiency of drone deliveries by easing their workload and extending their operational range, all while maintaining the main vision of the software solution.

The project successfully explored the potential of using drones for last-mile delivery of medical supplies, specifically test kits, in rural healthcare, with a focus on optimizing delivery routes via algorithmic solutions. The proposed solution effectively addressed the problem of finding the shortest route for drone deliveries under controlled conditions, thus setting a foundation for integrating drones into healthcare logistics.

This program is capable of reading a CSV file containing coordinate points, that represent delivery points. The program calculates the Euclidian distance between delivery points and stores the computed distances as edge weights in an adjacency matrix. This “distance matrix” is applied in the Branch and Bound algorithm to find the optimal route that minimizes the total travel distance, from a start node and through every other node exactly once and then back to starting node.

While the goals were met, the proposed solution had some problems, such as scaling and not accounting for real-world factors like terrain, weather, and battery life. To tackle these issues, future work could explore alternative approaches like Ant Colony Optimization for handling larger datasets and include features like using charging stations or support trucks to expand the delivery range.

The program’s wide range of potential applications, from food distribution to emergency logistics, demonstrates its flexibility. In addition to this, future developments of the software could incorporate features like graphical outputs, and improved usability options, such as progress indicators, which could enhance both its reliability and user experience.

In conclusion, this project provides a proof of concept for leveraging drones in healthcare logistics, offering an innovative solution to improve access to critical healthcare in rural areas.

- [1] P. Butterworth-Hayes, “Comparing the cost-effectiveness of drones v ground vehicles for medical, food and parcel deliveries,” Nov. 2019, Accessed: Oct. 24, 2024. [Online]. Available: <https://www.unmannedairspace.info/commentary/comparing-the-cost-effectiveness-of-drones-v-ground-vehicles-for-medical-food-and-parcel-deliveries/>
- [2] U. C. Bureau, “Urban and Rural.” Accessed: Oct. 30, 2024. [Online]. Available: <https://www.census.gov/programs-surveys/geography/guidance/geo-areas/urban-rural.html>
- [3] N. Douthit, S. Kiv, T. Dwolatzky, and S. Biswas, “Exposing some important barriers to health care access in the rural USA,” *Public Health*, vol. 129, no. 6, pp. 611–620, Jun. 2015, doi: [10.1016/j.puhe.2015.04.001](https://doi.org/10.1016/j.puhe.2015.04.001).
- [4] R. J. Buchanan, A. Stuifbergen, B. J. Chakravorty, S. Wang, L. Zhu, and M. Kim, “Urban/Rural Differences in Access and Barriers to Health Care for People with Multiple Sclerosis,” *Journal of Health and Human Services Administration*, vol. 29, no. 3, pp. 360–375, 2006, Accessed: Oct. 30, 2024. [Online]. Available: <https://www.jstor.org/stable/25790698>
- [5] E. C. Ziller, N. J. Anderson, and A. F. Coburn, “Access to Rural Mental Health Services: Service Use and Out-of-Pocket Costs,” *The Journal of Rural Health*, vol. 26, no. 3, pp. 214–224, 2010, doi: [10.1111/j.1748-0361.2010.00291.x](https://doi.org/10.1111/j.1748-0361.2010.00291.x).
- [6] E. Flaherty and S. J. Bartels, “Addressing the Community-Based Geriatric Healthcare Workforce Shortage by Leveraging the Potential of Interprofessional Teams,” *Journal of the American Geriatrics Society (JAGS)*, vol. 67, no. S2, pp. S400–S408, 2019.
- [7] J. Liu, Y. Ding, R. Qiu, Z. Meng, D. Sun, and X. Peng, “Drone-Assisted Long-Distance Delivery of Medical Supplies with Recharging Stations in Rural Communities,” *Health & Social Care in the Community*, vol. 2024, no. 1, p. 9143099, 2024, doi: [10.1155/2024/9143099](https://doi.org/10.1155/2024/9143099).
- [8] A. Toor, “This startup is using drones to deliver medicine in Rwanda,” Apr. 2016, Accessed: Oct. 28, 2024. [Online]. Available: <https://www.theverge.com/2016/4/5/11367274/zipline-drone-delivery-rwanda-medicine-blood>
- [9] J. M. Souch and J. S. Cossman, “A Commentary on Rural-Urban Disparities in COVID-19 Testing Rates per 100,000 and Risk Factors,” *The Journal of Rural Health*, vol. 37, no. 1, pp. 188–190, 2021, doi: [10.1111/jrh.12450](https://doi.org/10.1111/jrh.12450).
- [10] J. T. Mueller, K. McConnell, and K. Pofahl, “Impacts of the COVID-19 pandemic on rural America.” Accessed: Oct. 30, 2024. [Online]. Available: <https://www.pnas.org/doi/10.1073/pnas.2019378118>

- [11] U. D. of Homeland Security, “Federal Support to Expand National Testing Capabilities,” [Online]. Available: <https://www.fema.gov/fact-sheet/federal-support-expand-national-testing-capabilities>
- [12] M. F. M. at el., “Community-Based Testing Sites for SARS-CoV-2 — United States, March 2020–November 2021,” [Online]. Available: <https://www.cdc.gov/mmwr/volumes/70/wr/mm7049a3.htm>
- [13] T. R. Mercer and M. Salit, “Testing at scale during the COVID-19 pandemic,” *Nature Reviews Genetics*, vol. 22, no. 7, pp. 415–426, Jul. 2021, doi: [10.1038/s41576-021-00360-w](https://doi.org/10.1038/s41576-021-00360-w).
- [14] S. Leone, “How Drones Can Be Used to combat COVID-19,” [Online]. Available: <https://www.unicef.org/supply/media/5286/file/%20Rapid-guidance-how-can-drones-help-in-Covid-19-response.pdf.pdf>
- [15] Y. Vakulenko, P. Shams, D. Hellström, and K. Hjort, “Service innovation in e-commerce last mile delivery: Mapping the e-customer journey,” *Journal of business research*, vol. 101, pp. 461–468, 2019.
- [16] United Nations, *The World’s Cities in 2016*. in Statistical Papers - United Nations (Ser. A), Population and Vital Statistics Report. UN, 2016. doi: [10.18356/8519891f-en](https://doi.org/10.18356/8519891f-en).
- [17] D. Carroll, “Research suggests drones more efficient for last-mile deliveries,” Accessed: Oct. 23, 2024. [Online]. Available: <https://engineering.cmu.edu/news-events/news/2022/09/16-last-mile-drones.html>
- [18] X. Li, J. Tupayachi, A. Sharmin, and M. Martinez Ferguson, “Drone-Aided Delivery Methods, Challenge, and the Future: A Methodological Review,” *Drones (Basel)*, vol. 7, no. 3, p. 191–, 2023.
- [19] “Package Delivery by Drone (Part 135) | Federal Aviation Administration.” Accessed: Oct. 25, 2024. [Online]. Available: https://www.faa.gov/uas/advanced_operations/package_delivery_drone
- [20] “What Is the Part 135 Certificate and How Are Companies Like Flytrex and DroneUp Scaling Drone Delivery without One?,” [Online]. Available: <https://uavcoach.com/part-135/>
- [21] “Package Delivery by Drone (Part 135),” [Online]. Available: https://www.faa.gov/uas/advanced_operations/package_delivery_drone
- [22] “S.1939 - FAA Reauthorization Act of 2024,” [Online]. Available: <https://www.congress.gov/bill/118th-congress/senate-bill/1939/summary/25>
- [23] B. Aydin, “Public acceptance of drones: Knowledge, attitudes, and practice,” *Technology in Society*, vol. 59, p. 101180, 2019, doi: <https://doi.org/10.1016/j.techsoc.2019.101180>.
- [24] “Delivery drones will be a point of attraction in 2024,” 2024, [Online]. Available: <https://dataconomy.com/2024/01/03/delivery-drones-2024/>
- [25] V. Oksman and M. Kulju, “Towards Sustainable Mobility - Public Acceptance of Automated Last-Mile Deliveries,” pp. 257–266.
- [26] “Fundamentals of graph theory.” Accessed: Nov. 11, 2024. [Online]. Available: <https://www.geeksforgeeks.org/fundamentals-of-graph-theory/>

- [27] “Adjacency matrix representation.” Accessed: Nov. 11, 2024. [Online]. Available: <https://www.geeksforgeeks.org/adjacency-matrix/>
- [28] G. B. Dantzig and J. H. Ramser, “The Truck Dispatching Problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959, Accessed: Oct. 18, 2024. [Online]. Available: <http://www.jstor.org/stable/2627477>
- [29] M. M. Flood, “The Traveling-Salesman Problem,” *Operations Research*, vol. 4, no. 1, pp. 61–75, 1956, Accessed: Oct. 18, 2024. [Online]. Available: <http://www.jstor.org/stable/167517>
- [30] R. Sharda, B. Golden, S. Raghavan, S. Voss, and E. Wasil, *The Vehicle Routing Problem: Latest Advances and New Challenges*, Elektronisk udgave. in Operations Research/Computer Science Interfaces, 43. Boston, MA: Springer Science+Business Media, LLC, 2008.
- [31] GeeksForGeeks.org, “Traveling Salesman Problem using Branch And Bound.” Accessed: Oct. 30, 2024. [Online]. Available: <https://www.geeksforgeeks.org/traveling-salesman-problem-using-branch-and-bound-2/>
- [32] altexsoft, “Functional and Nonfunctional Requirements Specification.” Accessed: Nov. 11, 2024. [Online]. Available: <https://www.altexsoft.com/blog/functional-and-non-functional-requirements-specification-and-types/>
- [33] T. Kravchenko, T. Bogdanova, and T. Shevgunov, “Ranking Requirements Using MoSCoW Methodology in Practice,” in *Cybernetics Perspectives in Systems*, R. Silhavy, Ed., Cham: Springer International Publishing, 2022, pp. 188–199.
- [34] T. S. Utzle and M. Dorigo, “Traveling Salesman Problem,” [Online]. Available: <https://faculty.washington.edu/paymana/swarm/stutzle99-eaecs.pdf>
- [35] “BEYOND Program.” Accessed: Nov. 04, 2024. [Online]. Available: <https://cnoaa.com/beyond-program/>
- [36] “BEYOND | Federal Aviation Administration.” Accessed: Nov. 04, 2024. [Online]. Available: https://www.faa.gov/uas/programs_partnerships/beyond
- [37] “Adjacency List Representation.” Accessed: Oct. 28, 2024. [Online]. Available: <https://www.geeksforgeeks.org/adjacency-list-meaning-definition-in-dsa/>
- [38] “Dijkstra’s Algorithm for Adjacency List Representation | Greedy Algo-8.” Accessed: Oct. 28, 2024. [Online]. Available: <https://www.geeksforgeeks.org/dijkstras-algorithm-for-adjacency-list-representation-greedy-algo-8/>
- [39] “Representing Graph using Edge List.” Accessed: Dec. 16, 2024. [Online]. Available: <https://youcademy.org/graph-with-edge-list/>

9.1 Example Breakdown of the Branch & Bound algorithm on TSP on a Graph

9.1.0.1 Exploring Paths from Node 0:

Path $0 \rightarrow 2$:

Update Current Weight: Add the cost of traveling from Node 0 to Node 2:

$$\text{current_weight}_{\text{updated}} = \text{current_weight} + \text{Travel Cost from Node A to B} = 0 + 15 = 15$$

Update/Recalculate Lower Bound: The lower bound is adjusted by subtracting the average of the smallest outgoing edge of Node 0 (10) and the smallest edge of Node 2 (15) from the initial bound.

$$\text{LB}_{\text{updated}} = \text{LB} - \frac{\text{Smallest edge cost of Node A} + \text{Smallest edge cost of Node B}}{2}$$

Calculated with values:

$$\text{LB}_{\text{updated}} = 97.5 - \frac{10 + 15}{2} = 97.5 - 12.5 = 85$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (initially ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 15 + 85 = 100$$

Since $100 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 2.

Recursive Call: Mark node 2 as visited and recursively evaluate the remaining nodes from this point.

9.1.0.2 Exploring Paths from Node 2:

Path 2 → 3:

Update Current Weight: Add the cost of traveling from Node 2 to Node 3:

$$\text{current_weight} = 15 + 20 = 35$$

Update/Recalculate Lower Bound: Subtract the average of the smallest outgoing edge of Node 2 (15) and the smallest edge of Node 3 (15) from the lower bound.

$$\text{LB}_{\text{updated}} = 85 - \frac{15 + 15}{2} = 85 - 15 = 70$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (still ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 35 + 70 = 105$$

Since $105 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 3.

Recursive Call: Mark node 3 as visited and recursively evaluate the remaining nodes from this point.

9.1.0.3 Exploring Paths from Node 3:

Path 3 → 5:

Update Current Weight: Add the cost of traveling from Node 3 to Node 5:

$$\text{current_weight} = 35 + 15 = 50$$

Update/Recalculate Lower Bound: Subtract the average of the smallest outgoing edge of Node 3 (15) and the smallest edge of Node 5 (10) from the lower bound.

$$\text{LB}_{\text{updated}} = 70 - \frac{15 + 10}{2} = 70 - 12.5 = 57.5$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (still ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 50 + 57.5 = 107.5$$

Since $107.5 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 5.

Recursive Call: Mark node 5 as visited and recursively evaluate the remaining nodes from this point.

9.1.0.4 Exploring Paths from Node 5:

Path 5 → 4:

Update Current Weight: Add the cost of traveling from Node 5 to Node 4:

$$\text{current_weight} = 50 + 10 = 60$$

Update/Recalculate Lower Bound: Subtract the average of the smallest outgoing edge of Node 5 (10) and the smallest edge of Node 4 (10) from the lower bound.

$$\text{LB}_{\text{updated}} = 57.5 - \frac{10 + 10}{2} = 57.5 - 10 = 47.5$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (still ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 60 + 47.5 = 107.5$$

Since $107.5 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 4.

Recursive Call: Mark node 4 as visited and recursively evaluate the remaining nodes from this point.

9.1.0.5 Exploring Paths from Node 4:

Path 4 → 6:

Update Current Weight: Add the cost of traveling from Node 4 to Node 6:

$$\text{current_weight} = 60 + 10 = 70$$

Update/Recalculate Lower Bound: Subtract the average of the smallest outgoing edge of Node 4 (10) and the smallest edge of Node 6 (10) from the lower bound.

$$\text{LB}_{\text{updated}} = 47.5 - \frac{10 + 10}{2} = 47.5 - 10 = 37.5$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (still ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 70 + 37.5 = 107.5$$

Since $107.5 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 6.

Recursive Call: Mark node 6 as visited and recursively evaluate the remaining nodes from this point.

9.1.0.6 Exploring Paths from Node 6:

Path 6 → 1:

Update Current Weight: Add the cost of traveling from Node 6 to Node 1:

$$\text{current_weight} = 70 + 25 = 95$$

Update/Recalculate Lower Bound: Subtract the average of the smallest outgoing edge of Node 6 (10) and the smallest edge of Node 1 (10) from the lower bound.

$$\text{LB}_{\text{updated}} = 37.5 - \frac{10 + 10}{2} = 37.5 - 10 = 27.5$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (still ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 95 + 27.5 = 122.5$$

Since $122.5 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 1.

Recursive Call: Mark node 1 as visited and recursively evaluate the remaining nodes from this point.

9.1.0.7 Exploring Paths from Node 1:

Path 1 → 0:

Update Current Weight: Add the cost of traveling from Node 1 to Node 0:

$$\text{current_weight} = 95 + 10 = 105$$

Update/Recalculate Lower Bound: Subtract the average of the smallest outgoing edge of Node 1 (10) and the smallest edge of Node 0 (10) from the lower bound.

$$\text{LB}_{\text{updated}} = 27.5 - \frac{10 + 10}{2} = 27.5 - 10 = 17.5$$

Pruning Check: If the sum of $\text{current_weight} + \text{LB}_{\text{updated}}$ is greater than final_result (still ∞), this path is pruned. Otherwise, it is explored further:

$$\text{Potential Total Cost} = \text{current_weight} + \text{LB}_{\text{updated}} = 105 + 17.5 = 122.5$$

Since $122.5 < \text{final_result} (\infty)$, this path is valid, which means proceed to Node 0.

Recursive Call: Mark node 0 as visited and recursively evaluate the remaining nodes from this point, which should result in the path/route completing because every node has been “walked” through exactly once, except node 0, because it is the starting and end point, due to the Traveling Salesman Problem.

9.2 Part 135 certificate tiers

- Tier 1: Part 135 Single pilot is a certificate holder that has various limitations, such as craft-size and personnel size. The personnel size is limited to only. That being the certificate holder himself. These limitations are for all part 135 operations [19].
- Tier 2: Part 135 Single pilot in command is also a limited part 135 certificate, but does have more loose restrictions. The looser restrictions include one pilot in command and three seconds pilots in command. There are also still limitations and restrictions on the craft-size and scope of the part 135 operations [19].
- Tier 3: A Basic part 135 certificate is also a limited part 135 certificate in the size and scope of the operations. The limitations include a maximum five pilots, including seconds pilots. A maximum of five aircraft can be used in their part 135 operations [19].
- Tier 4: A standard part 135 certificate is no longer a limited certificate, and therefore has no limitations on scope, personnel-size or craft-size. However the certificate-holder must be granted authorizations for each type of operation they would want to conduct [19].

9.3 General rules for operating a UAS for both commercial recreational use

For recreational use:

- You must fly at or below 400 ft (~122 meters)
- All drones registered must weigh below 55 lbs (25kg). However, this rule is voided with a part 135 certificate.
- Download and use B4UFLY Mobile App. The app from the FAA, which uses your GPS location to provide real-time information about airspace restrictions and other flying requirements.
- You must be aware of No Drone Zones
- You must be within the visual line of sight of the UAS so that you can see the drone at all times. This is also voided with a part 135 standard certificate and/or participation in the BEYOND program, which is a program established Choctaw Nation of Oklahoma (CNO) that allows drone operators fly drones from outside visual line of sight[35], [36].
- You must learn about airspace restrictions, especially around airports, so your drone does not endanger people or other aircraft.
- Give way to and do not interfere with other aircraft.

For commercial use:

- You must obtain a Remote Pilot Certificate, or a part 107 liscence.
- Pass the initial aeronautical knowledge exam: “Unmanned Aircraft General – Small (UAG)”
- Be able to read, write, speak and understand english.
- Be atleast 16 years old.
- Be mentally and physically able to operate a UAS
- Create an IACRA (Integrated Airman Certification and Rating Application) profile and obtain an FAA Tracking Number (FTN).
- You are allowed to fly over people and other moving vehicles at night without a waiver under Part 107 as long as the rules are followed. You must have airspace authorization for night operations.

9.4 Graph Representation Using Adjacency Lists

9.4.1 What is an Adjacency List?

An adjacency list is a way of representing a graph in which each node has a list of its neighboring nodes, which refer to all the nodes that the given node is connected to by an edge. In this graph representation, the graph is stored as an array or list of lists. Each element of the array corresponds to a node and the associated list, which stores the neighbors of that given node [37]. Figure 36 shows an example graph representation using an adjacency list. The adjacency list shows neighboring vertices for each vertex in the graph. Vertex 1 is connected to vertex 2, 3, and 4, whereas vertex 4 is only connected to vertex 1.

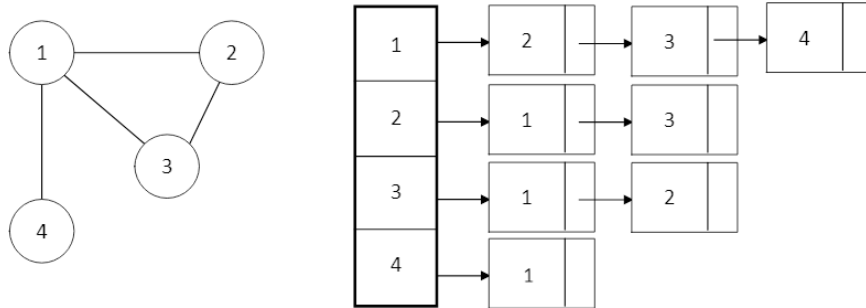


Figure 36: Example Adjacency List [37].

9.4.2 Memory Usage

The memory usage of an adjacency list depends on the number of nodes and edges in the graph. For a complete graph with n nodes, each node will have $n - 1$ neighbors, which results in $n(n - 1)$ entries across all list. Hence the total memory usage is $O(n^2)$, which is the same as the adjacency matrix for a complete graph as shown in Section 4.2.2.1. However in sparse graphs (non-complete graphs), adjacency lists are more memory efficient, because each node only stores its actual neighbors, reducing the space required [38].

9.4.3 Time Complexity for Edge Lookups

The time complexity for edge lookups in an adjacency list is $O(n)$ in the worst case, because to determine if there is an edge between two nodes, it has to traverse the list of neighbors for one of the nodes. In a complete graph, since every node has $n - 1$ neighbors, the list for each node will have $n - 1$ entries, meaning that the lookup time grows linearly with n , thus $O(n)$. This is less efficient compared to the constant time access ($O(1)$) provided by an adjacency matrix, as shown in Section 4.2.2.1.

9.5 Graph Representation Using Edge Lists

9.5.1 What is an Edge List?

An edge list is a simple way to represent a graph, where each edge is stored as a pair of nodes. This representation simply lists all edges in the graph, with each edge being represented as a pair of nodes (i, j) , indicating that there is an edge between node i and j [39].

9.5.2 Memory Usage

For a complete graph with n nodes the number of edges is $\frac{n(n-1)}{2}$, given that each edge is stored only once in an undirected graph. The memory usage for an edge list in a complete graph is $O(n^2)$, as it stores all the edges directly. While this is the same as the memory usage of both adjacency matrix and adjacency list for a complete graph, as shown in Section 4.2.2.1, it is more space-efficient in sparse graphs, where the number of edges is much lower.

9.5.3 Time Complexity for Edge Lookups

The time complexity for edge lookups in an edge list is $O(n^2)$ in the worst case. This is because the edge list contains all the edges of the graph and thus must scan through the entire list to find a specific edge. In a complete graph, this means that in the worst case it has to traverse all $O(n^2)$ edges to locate the desired edge. This makes edge lists inefficient for algorithms that require constant time access to edges, such as the Branch and Bound method for the Traveling Salesman Problem (TSP).

9.6 Random node generation

This section is a quick walkthrough of a small program for generating a file with random node coordinates.

The function in Figure 37 generates a specified number of coordinate sets, n , with a given max size given as *size* in the function call. Coordinates sets are printed to the same input file as the pathfinding algorithm reads from in Section 5.5.2. A pointer to the file is given as an argument in the function.

First, the function validates the filepointer, returning an error to the user if the file is not found. Then the header rows of the file are printed using *fprintf()* and the random generation function *rand()* is seeded with the current time using *srand(time(NULL))*. Then *rand()* is used to return random double values from $0 \rightarrow \text{size}$, excluding 0 and size, and *fprintf()* to print them to the file.

```
8 void generate_addresses_float(const char* filepath, int n, int size) {
9     FILE* filepointer = fopen(filepath, mode: "w");
10
11     // If the file is not found, print to the user and EXIT
12     if (!filepointer) {
13         printf("Can't open file\n");
14         printf("filename %s", filepath);
15         exit(EXIT_FAILURE);
16     }
17     // Print the header of the file
18     fprintf(filepointer, "x-value, y-value \n");
19     srand(time(NULL));
20
21     // Generate the specified number of coordinate sets
22     for (int i = 0; i < n; i++) {
23         // Generates random float values from 0 -> size, (Size is specified in the function call)
24         double x = (float)rand()/(float)(RAND_MAX) * size;
25         double y = (float)rand()/(float)(RAND_MAX) * size;
26
27         // Prints coordinates to the file
28         fprintf(filepointer, "%.3lf,%.3lf \n", x,y);
29     }
30     fclose(filepointer);
31 }
```

Figure 37: Random Node Generation.

9.7 Brute Force Time Comparison

In this test case, the scenario is evaluated where a program uses brute force to find the shortest route for TSP. This method involves systematically exploring every possible route created with a given number of nodes

Example

Consider a complete graph with 10 nodes where each node is connected to every other node with a weighted edge. The brute force approach would need to calculate all possible permutations of these 10 nodes. This results in $10!$ (3,628,800 permutations) routes to evaluate.

Execution for a Theoretical Program:

Say the theoretical program iterates through all $10!$ routes, calculating the total distance for each route and comparing it to the current best route. Eventually, it will identify the optimal route with the shortest total distance. The time required depends on how many calculations the system can handle per second.

Analysis:

While brute force ensures the shortest route is found, its computational complexity makes it impractical for larger graphs. The number of permutations grows factorially as shown in Figure 38 below:

Number of Nodes (n)	Total Routes ($n!$)	Time Taken
10	3,628,800	0.003 seconds
12	479,001,600	0.479 seconds
14	87,178,291,200	87,18 seconds
20	2,432,902,008,176,640,000	77.1 years

Figure 38: Results of the analysis for 10, 12, 14, and 20 nodes

As n increases, the time required quickly becomes unmanageable.

How to Calculate Time Taken:

Assuming the computer can perform 10^9 calculations per second. To estimate the time taken to brute force all possible routes, calculate the total number of calculations involved and divide it by the number of calculations the computer can handle per second.

Each route requires a certain number of calculations to determine its total distance. Assume that evaluating each route requires C calculations, then the total number of calculations for $n!$ routes is:

$$\text{Total Calculations} = C * n!$$

Formula for the calculation of the time required to brute force all possible routes:

$$\text{Time Taken (second)} = \frac{C * n!}{10^9}$$

For simplicity and the sake of the example, let's assume that $C = 1$ calculation per route. The time taken for different values of $n!$ can be computed as follows:

$n = 10$:

For $n = 10$, where $10! = 3,628,800$ routes:

$$\text{Time Taken (second)} = \frac{3,628,800}{10^9} = 0.00363 \text{ seconds}$$

$n = 12$:

For $n = 12$, where $12! = 479,001,600$ routes:

$$\text{Time Taken (second)} = \frac{479,001,600}{10^9} = 0.479 \text{ seconds}$$

$n = 14$:

For $n = 14$, where $14! = 87,178,291,200$ routes:

$$\text{Time Taken (second)} = \frac{87,178,291,200}{10^9} = 87.18 \text{ seconds}$$

$n = 20$:

For $n = 20$, where $20! = 2,432,902,008,176,640,000$ routes:

$$\text{Time Taken (second)} = \frac{2,432,902,008,176,640,000}{10^9} = 2,432,902,008.18 \text{ seconds} = 77.1 \text{ years}$$

Why Brute Force is Inefficient:

The brute force approach is inefficient, due to the factorial growth of permutations, which result in every increase in the number of nodes leading to a great increase in computational time. The brute force approach also uses a lot of time calculating routes that are far worse than the optimal route, and unlike other algorithms, brute force does not exclude/prune suboptimal paths early in the process, leading to unnecessary calculations.

Brute Force vs Branch & Bound:

However, the Branch & Bound algorithm significantly reduces the number of routes evaluated. It does this by maintaining a bound for the shortest known route, the algorithm prunes routes that exceed this bound. If a partial route exceeds the current best cost, it does not explore further permutations of that route. This approach eliminates entire subsets of permutations, making it a lot faster in practice.

In conclusion, while brute force guarantees the correct solution, it is computationally expensive and impractical for larger graphs due to it not pruning suboptimal paths. This makes it a poor choice for solving TSP beyond very small nodes counts. The Branch & Bound algorithm, by strategically ignoring irrelevant routes, is a lot more efficient and scalable for finding the shortest path in TSP.